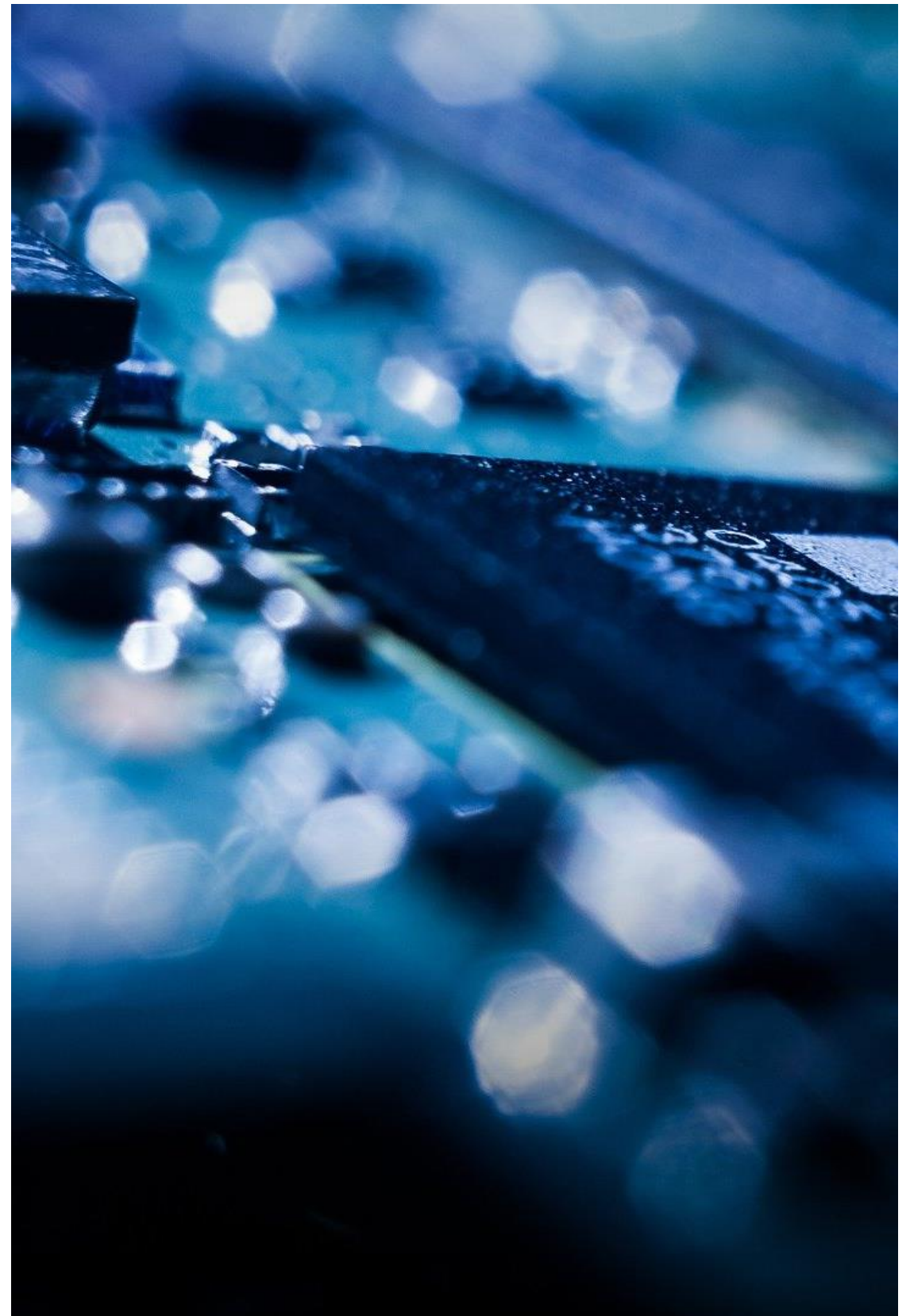


# Programmierung eingebetteter Systeme

Ausnahmereverarbeitung  
(Exception processing)

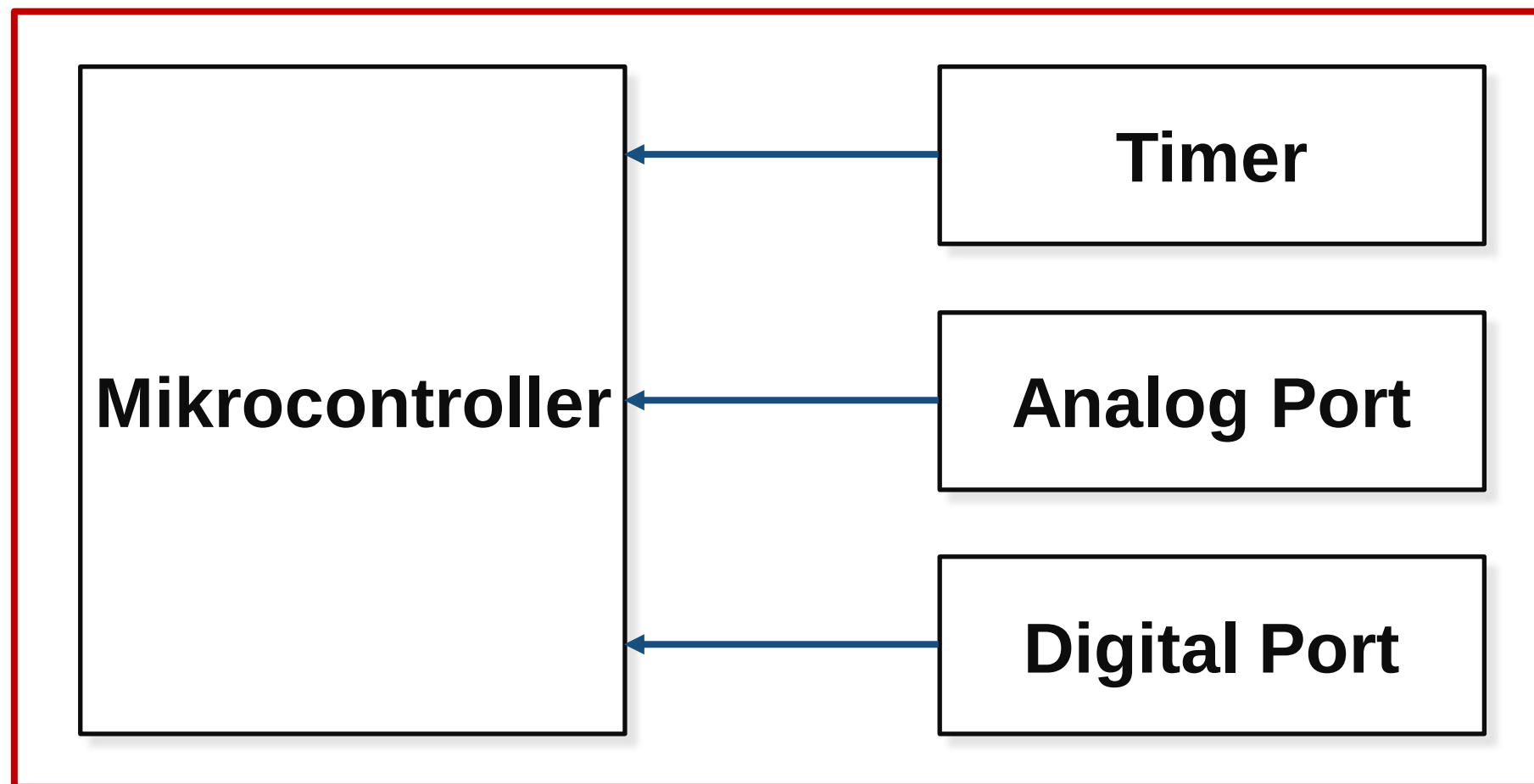
**Prof. Dr. Henning Trsek**

Technische Hochschule Ostwestfalen-Lippe  
Fachbereich Elektrotechnik und Technische Informatik  
inIT - Institut für industrielle Informationstechnik  
[henning.trsek@th-owl.de](mailto:henning.trsek@th-owl.de)



- ▶ Ausnahmereverarbeitung allgemein  
(Exception processing)
- ▶ Ausnahmereverarbeitung und Stackbenutzung

# Ausnahmereverarbeitung (Exceptions) *inIT*

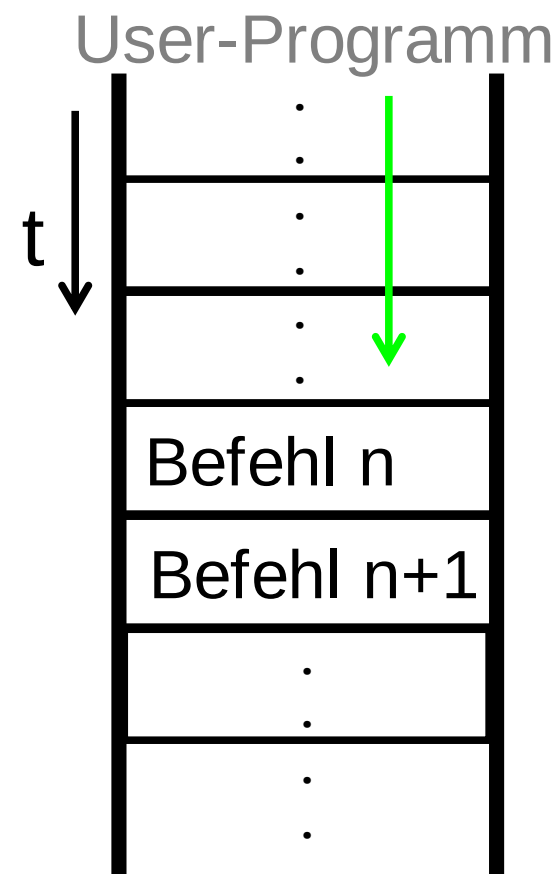


Unterbrechung des normalen Programmablaufs durch ein äusseres Ereignis

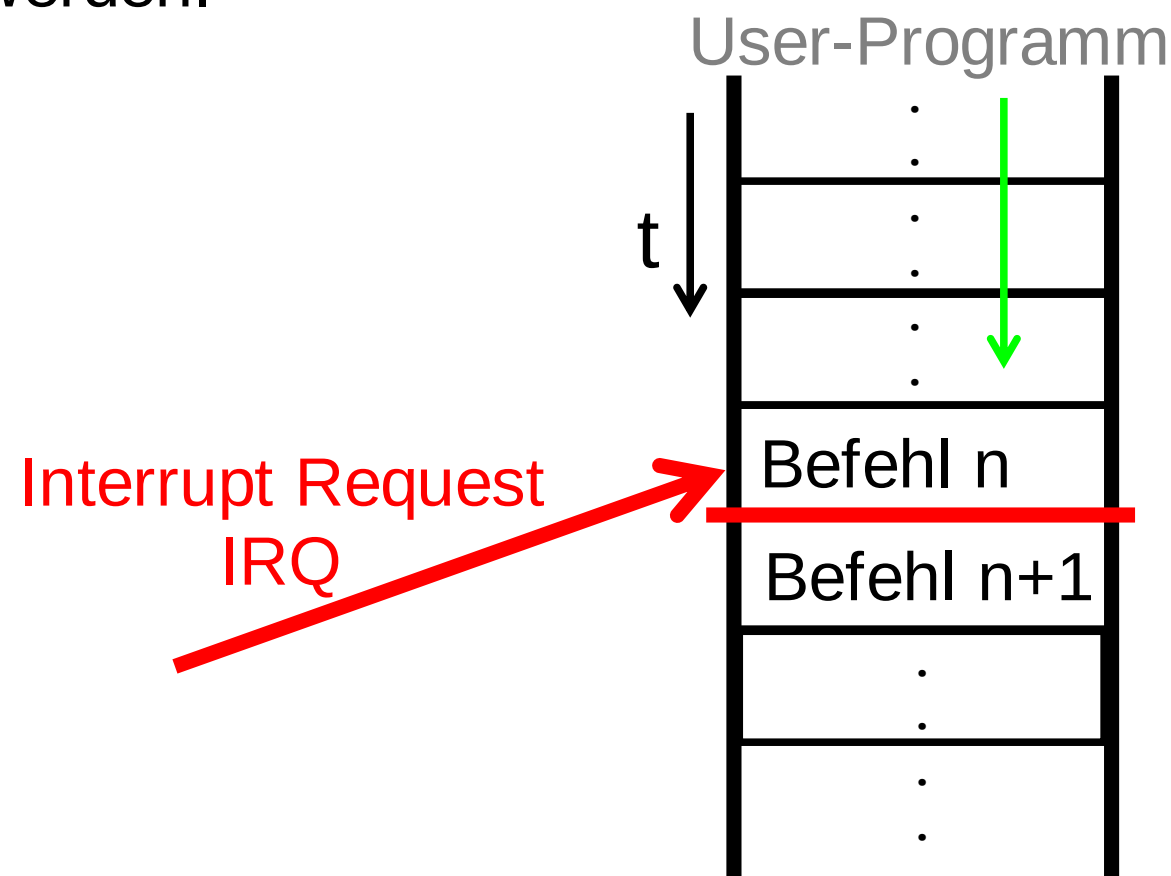
**Interrupt Request (IRQ)**

Das aktuell durch den Mikroprozessor bearbeitete Programm soll unterbrochen und ein anderes Programm (mit dem auf das äussere Ereignis reagiert wird) soll bearbeitet werden.

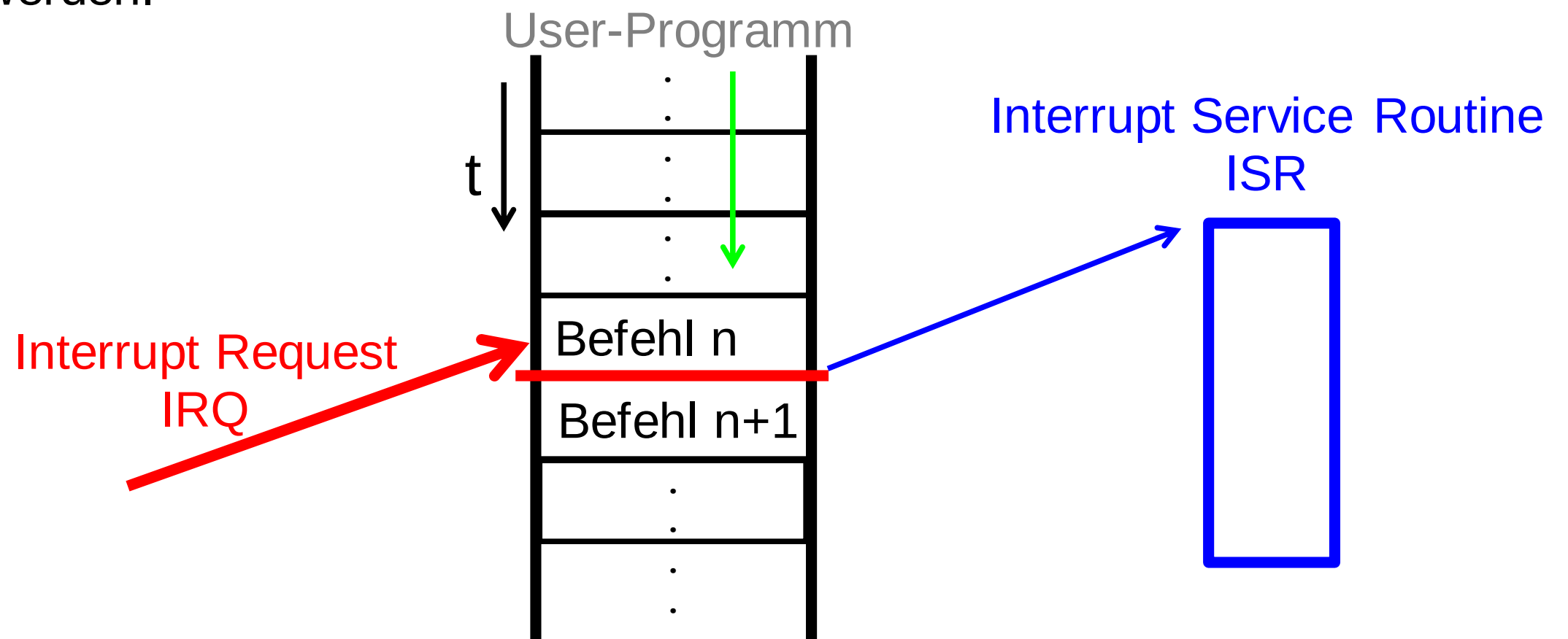
Das aktuell durch den Mikroprozessor bearbeitete Programm soll unterbrochen und ein anderes Programm (mit dem auf das äussere Ereignis reagiert wird) soll bearbeitet werden.



Das aktuell durch den Mikroprozessor bearbeitete Programm soll unterbrochen und ein anderes Programm (mit dem auf das äussere Ereignis reagiert wird) soll bearbeitet werden.



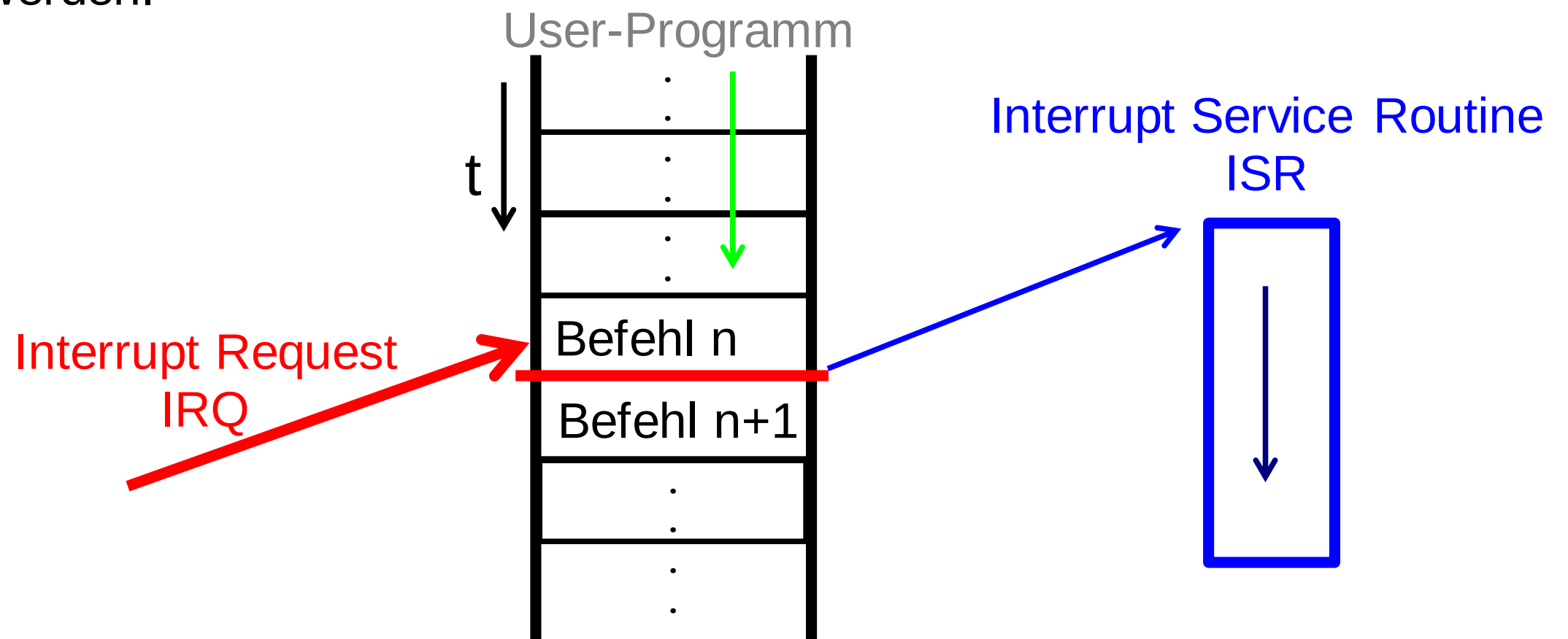
Das aktuell durch den Mikroprozessor bearbeitete Programm soll unterbrochen und ein anderes Programm (mit dem auf das äussere Ereignis reagiert wird) soll bearbeitet werden.





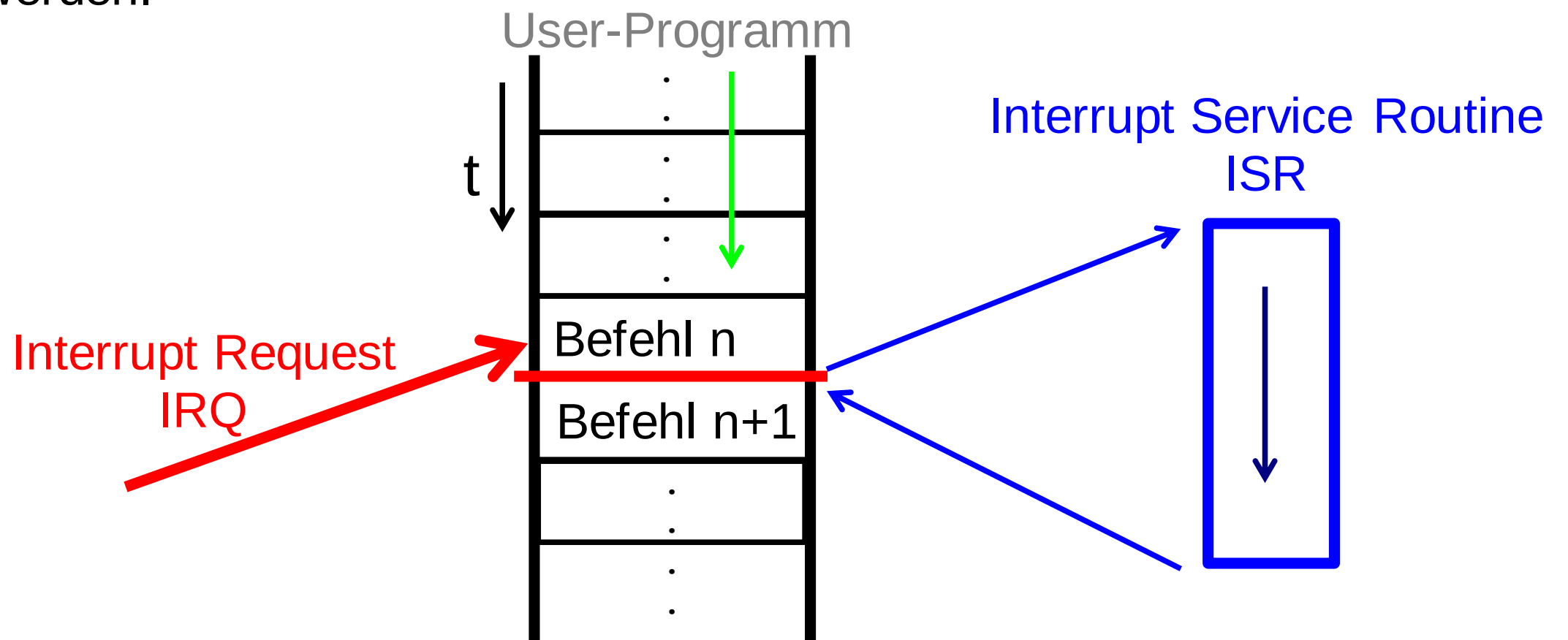
# Ausnahmeverarbeitung (Exceptions)

Das aktuell durch den Mikroprozessor bearbeitete Programm soll unterbrochen und ein anderes Programm (mit dem auf das äussere Ereignis reagiert wird) soll bearbeitet werden.

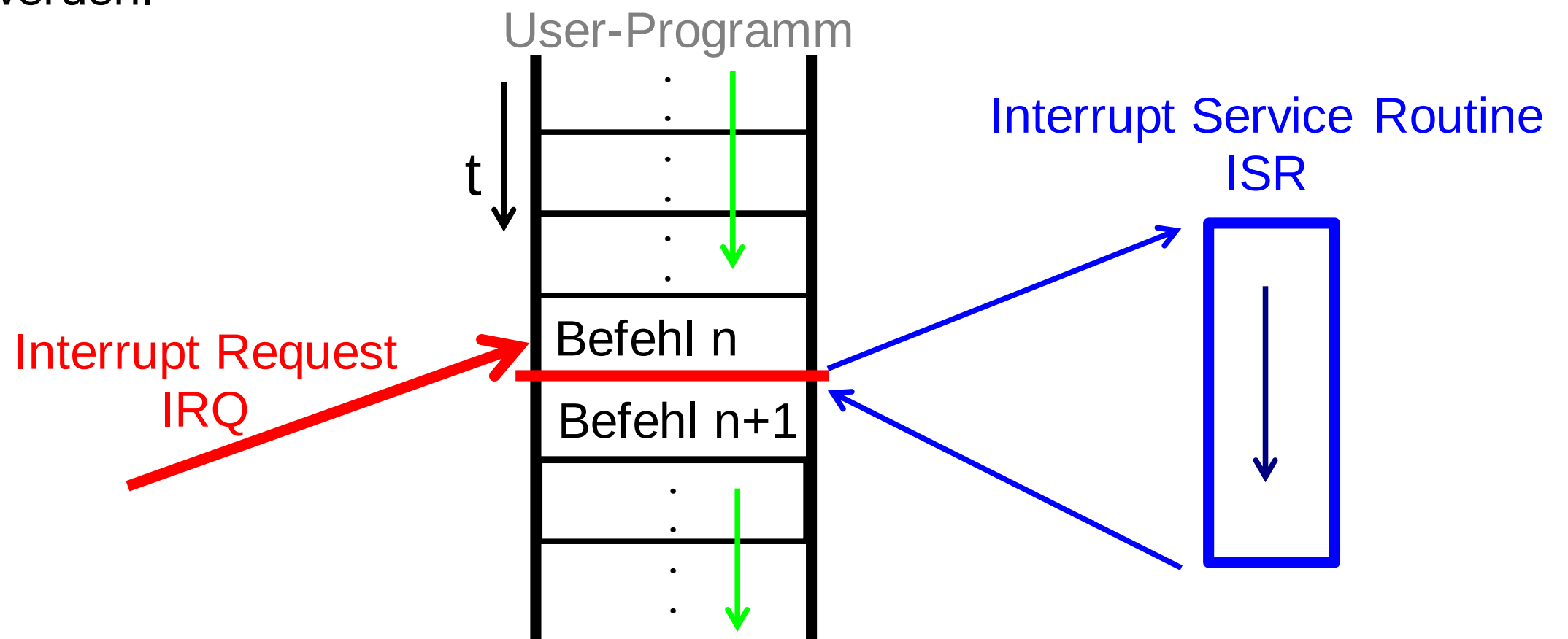




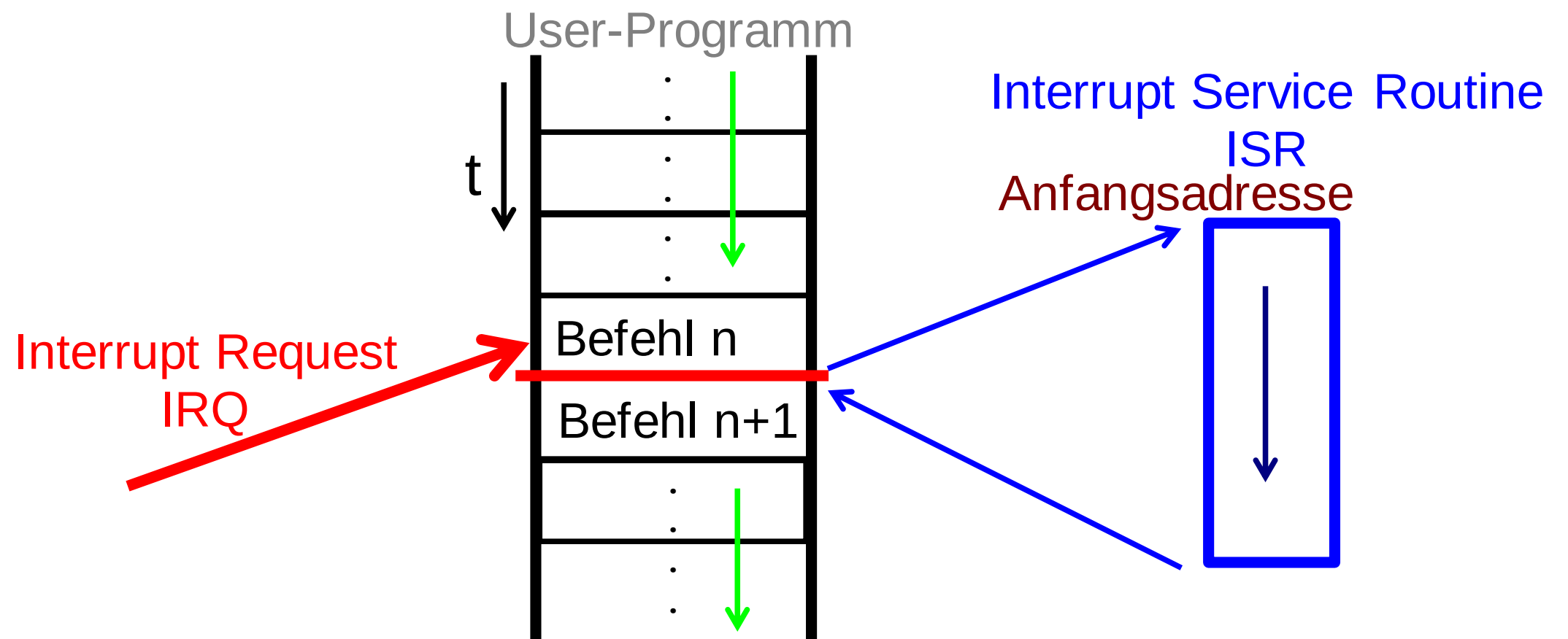
Das aktuell durch den Mikroprozessor bearbeitete Programm soll unterbrochen und ein anderes Programm (mit dem auf das äussere Ereignis reagiert wird) soll bearbeitet werden.



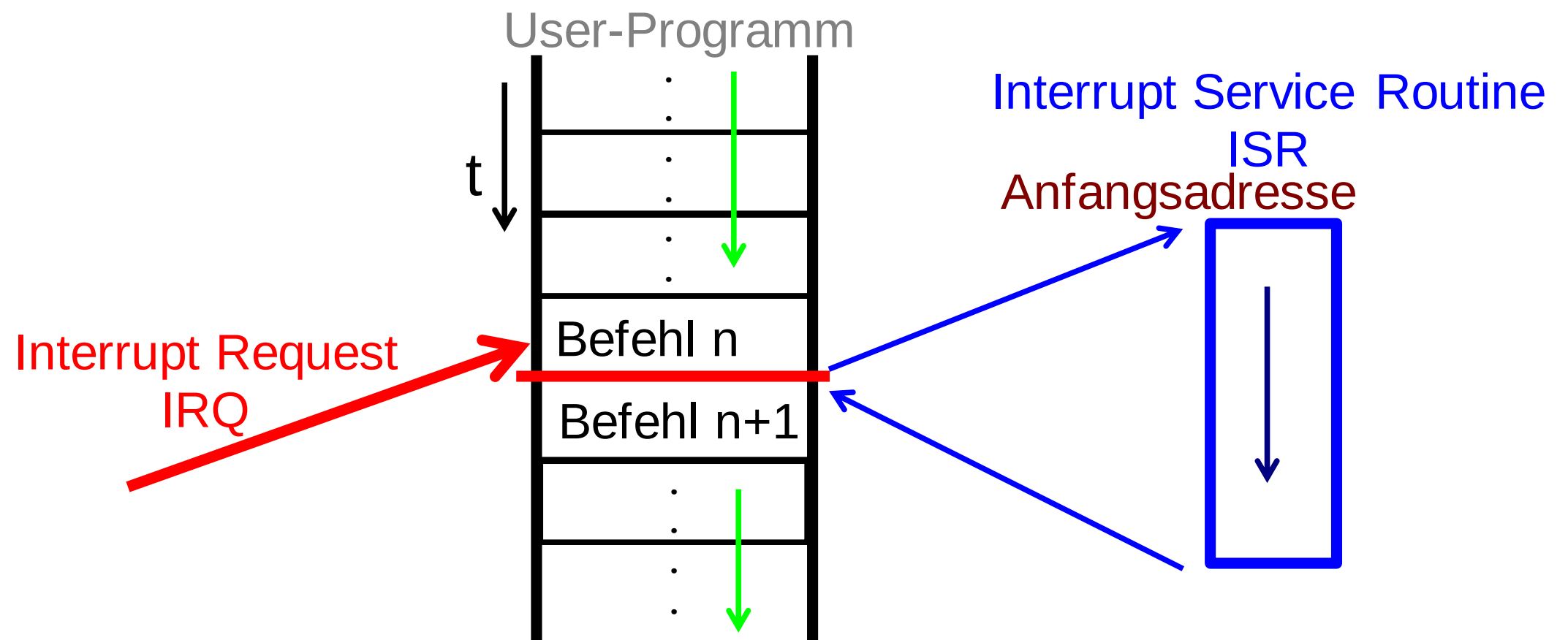
Das aktuell durch den Mikroprozessor bearbeitete Programm soll unterbrochen und ein anderes Programm (mit dem auf das äussere Ereignis reagiert wird) soll bearbeitet werden.



Woher kennt der Prozessor die Anfangsadresse des Programms, das aufgrund des IRQ ausgeführt werden soll ?

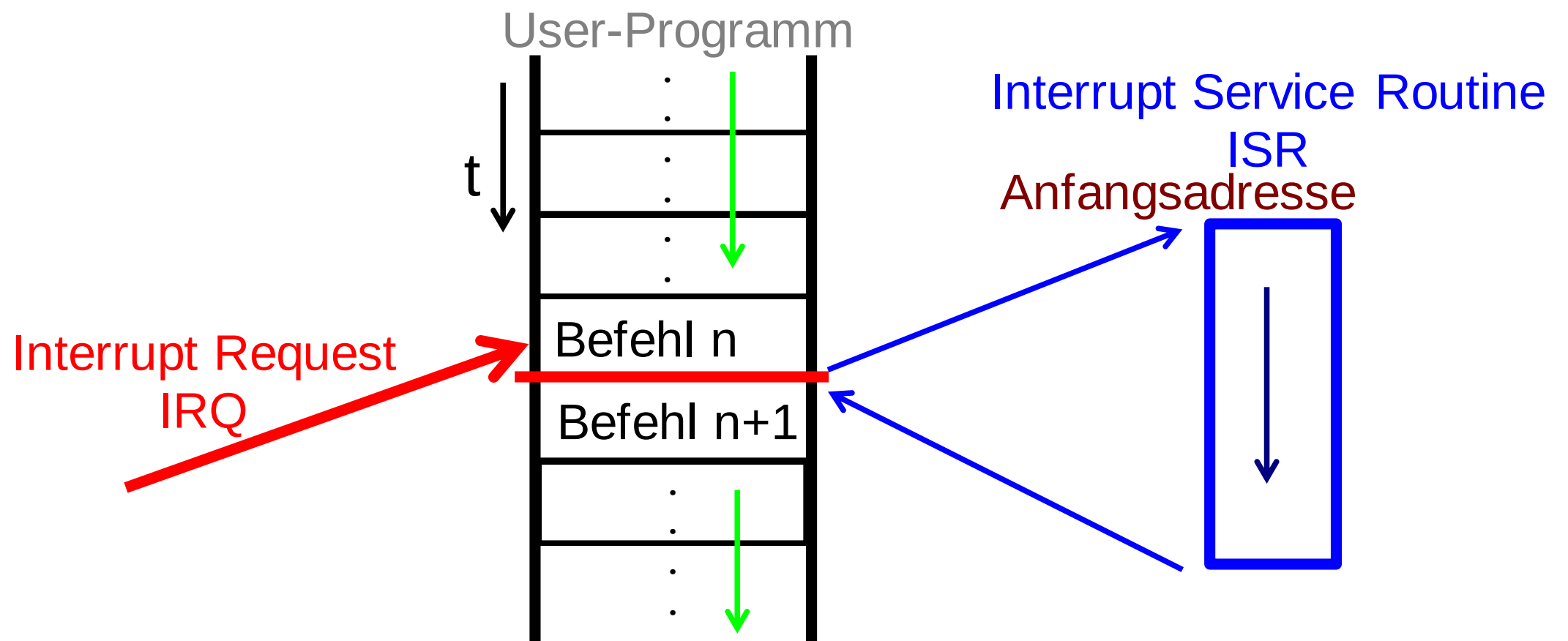


Woher kennt der Prozessor die Anfangsadresse des Programms, das aufgrund des IRQ ausgeführt werden soll ?



Im User-Programm kann die Information nicht vorhanden sein.

Woher kennt der Prozessor die Anfangsadresse des Programms, das aufgrund des IRQ ausgeführt werden soll ?



Im User-Programm kann die Information nicht vorhanden sein.

Die Information zu den Interrupt Service Routinen (allgemein: Exception Service Routinen) ist an einer zentralen Stelle abgelegt, die der Prozessor unabhängig vom gerade bearbeiteten Programm „kennt“.

# Exception Vector Table

Die zentrale Tabelle mit den Anfangsadressen aller Exception Service Routinen ist die Exception Vector Table.

| Exception number | IRQ number | Offset | Vector             |
|------------------|------------|--------|--------------------|
| 255              | 239        | 0x03FC | IRQ239             |
| ·                |            | ·      | ·                  |
| ·                |            | ·      | ·                  |
| ·                |            | ·      | ·                  |
| 18               | 2          | 0x004C | IRQ2               |
| 17               | 1          | 0x0048 | IRQ1               |
| 16               | 0          | 0x0044 | IRQ0               |
| 15               | -1         | 0x0040 | Systick            |
| 14               | -2         | 0x003C | PendSV             |
| 13               |            | 0x0038 | Reserved           |
| 12               |            |        | Reserved for Debug |
| 11               | -5         |        | SVCall             |
| 10               |            | 0x002C |                    |
| ...              |            |        |                    |

# Exception Vector Table

stm32: Die zentrale Tabelle mit den Anfangsadressen aller Exception Service Routinen ist die **Exception Vector Table**.

Speicher

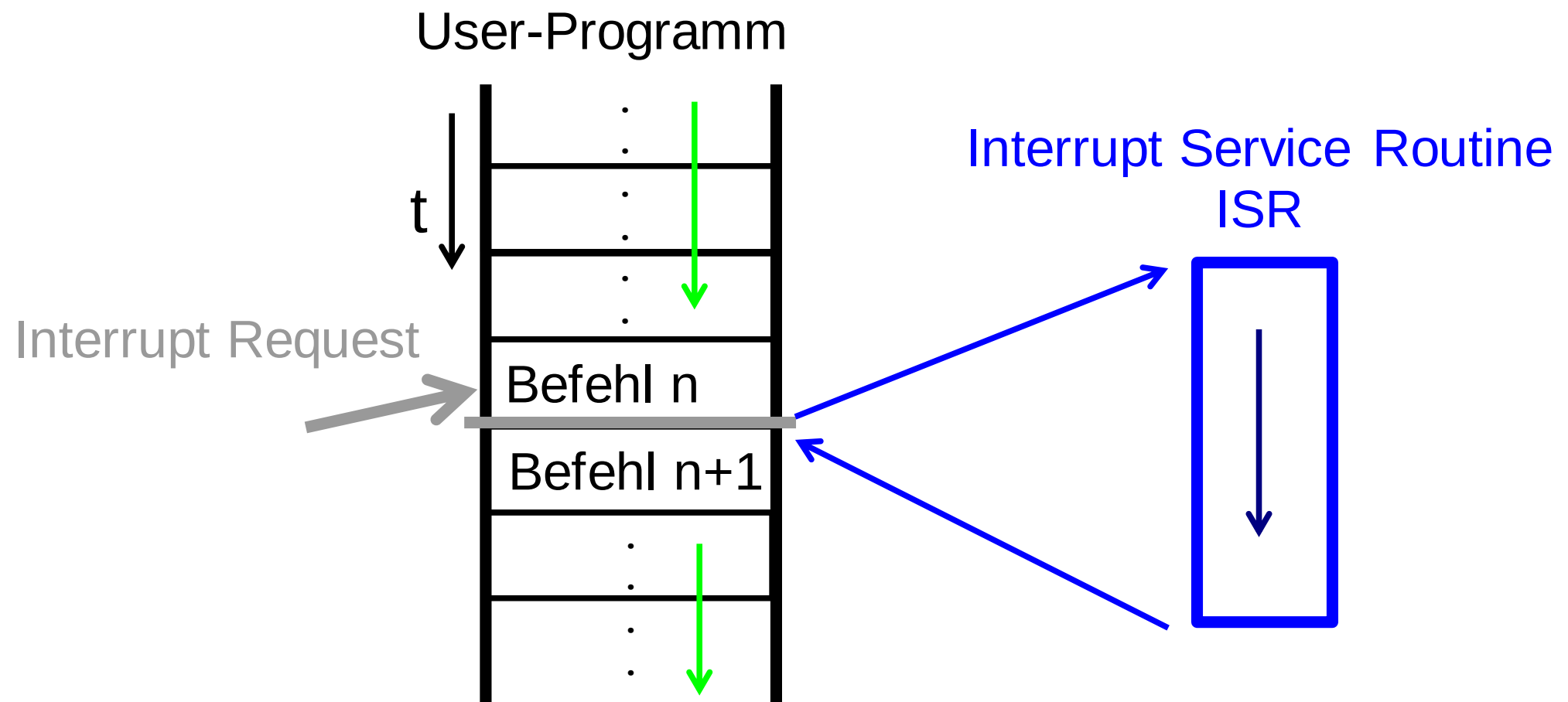
\$00000000

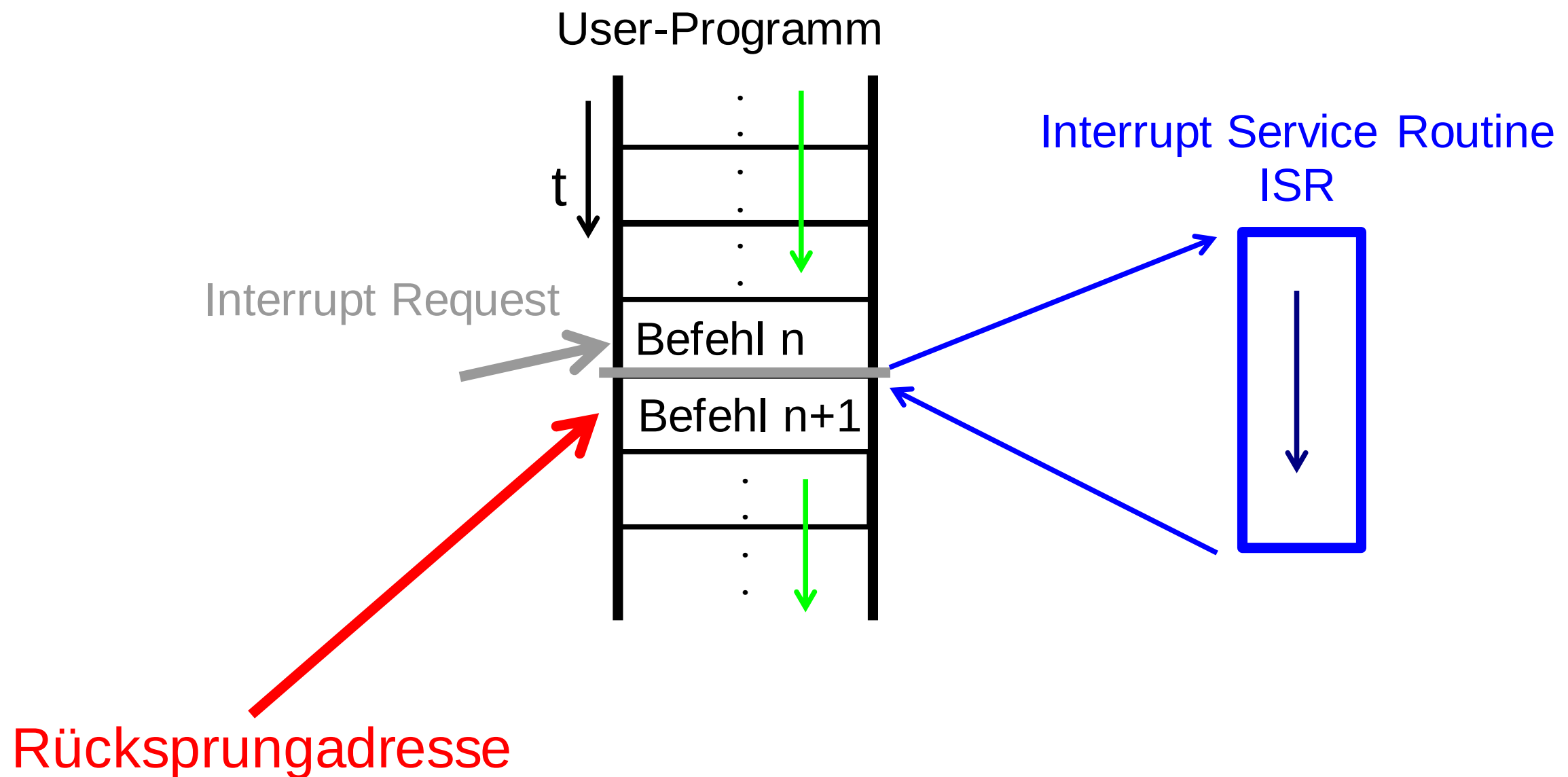
| Speicher-<br>adresse | Inhalt                                        |
|----------------------|-----------------------------------------------|
| \$00000000           | Adresse (Pointer auf ExceptionServiceRoutine) |
| \$00000004           | Adresse (Pointer auf ExceptionServiceRoutine) |
| \$00000008           | Adresse (Pointer auf ExceptionServiceRoutine) |
| ...                  | ...                                           |

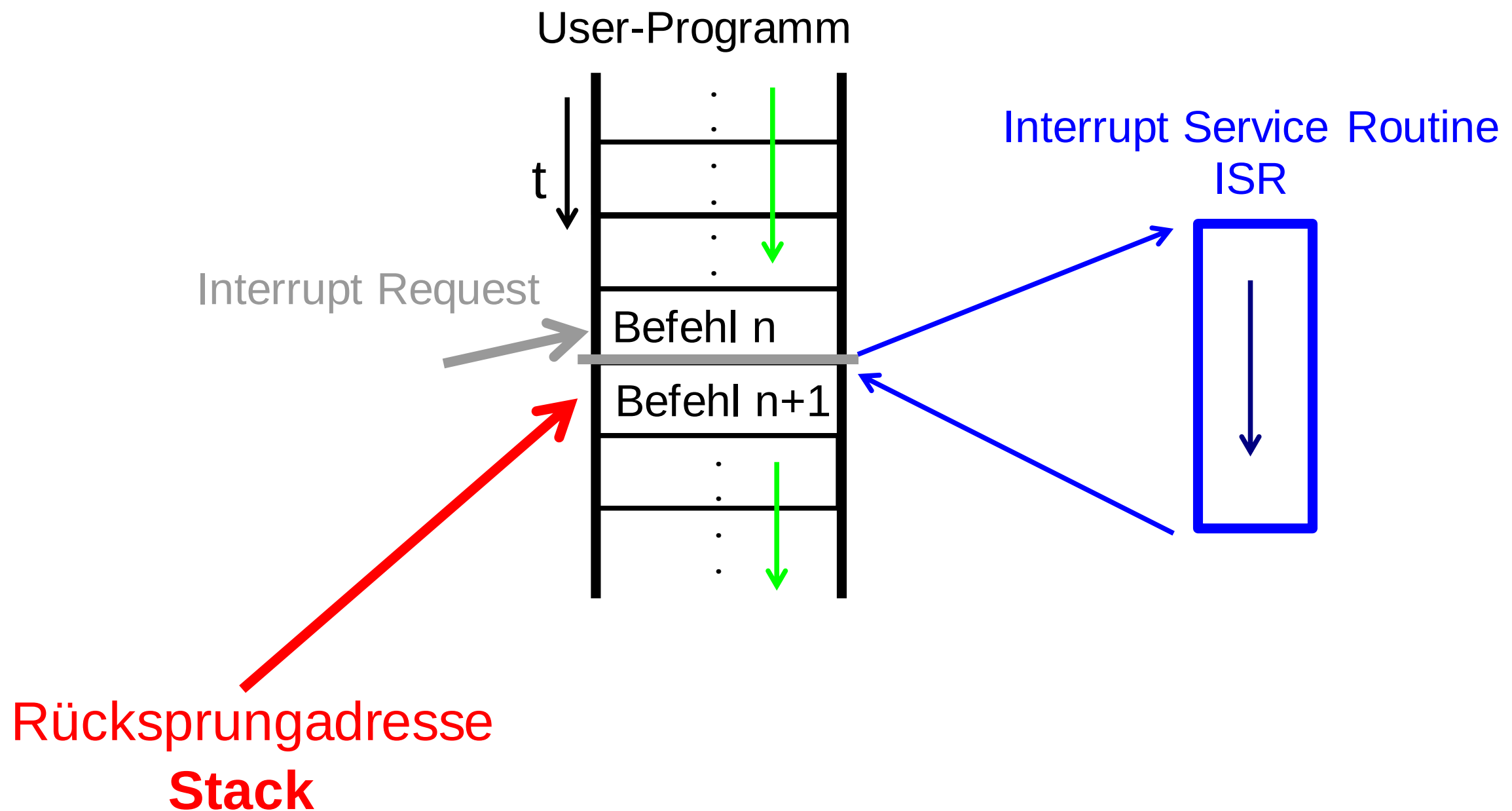


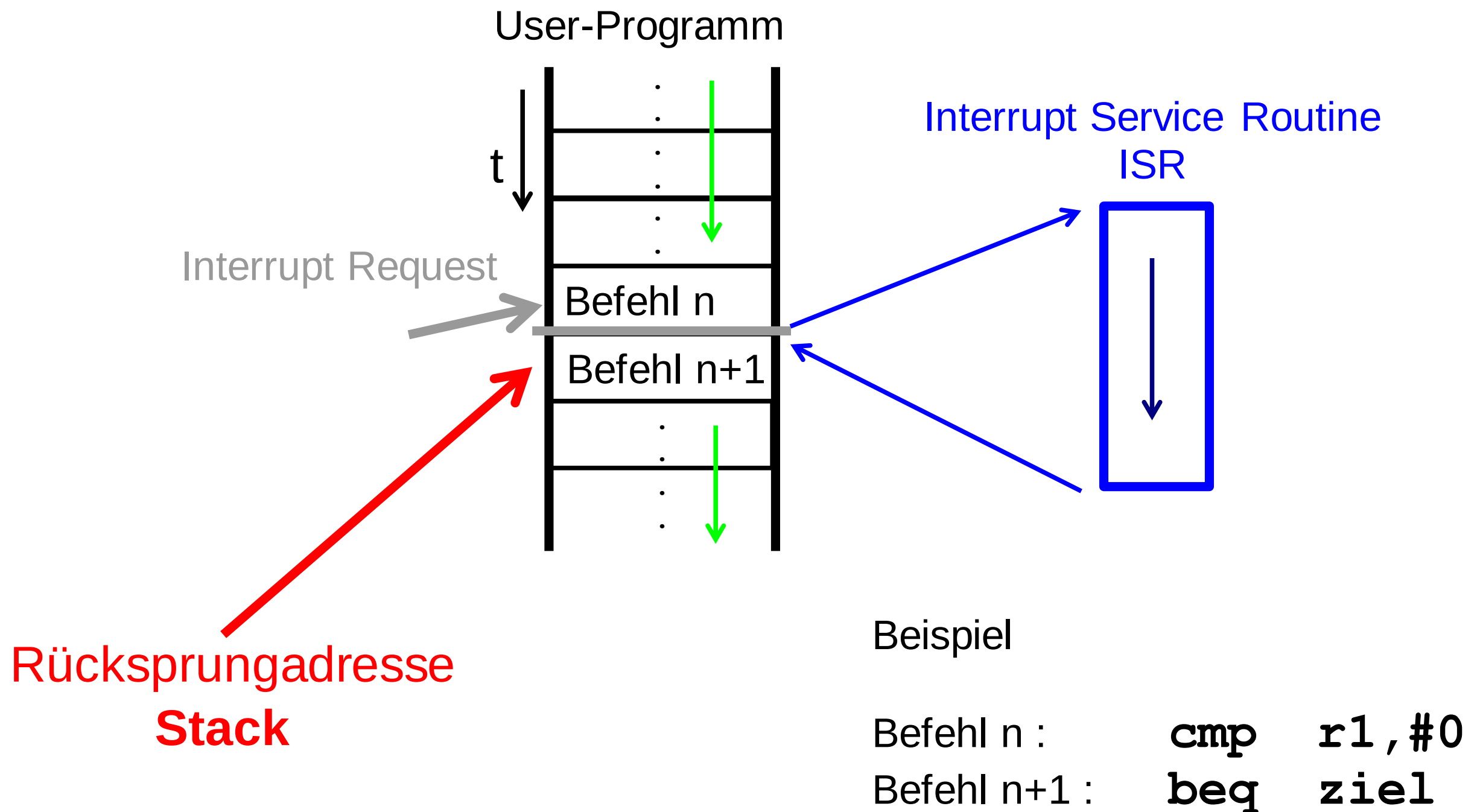
# Ausnahmeverarbeitung und Stackbenutzung

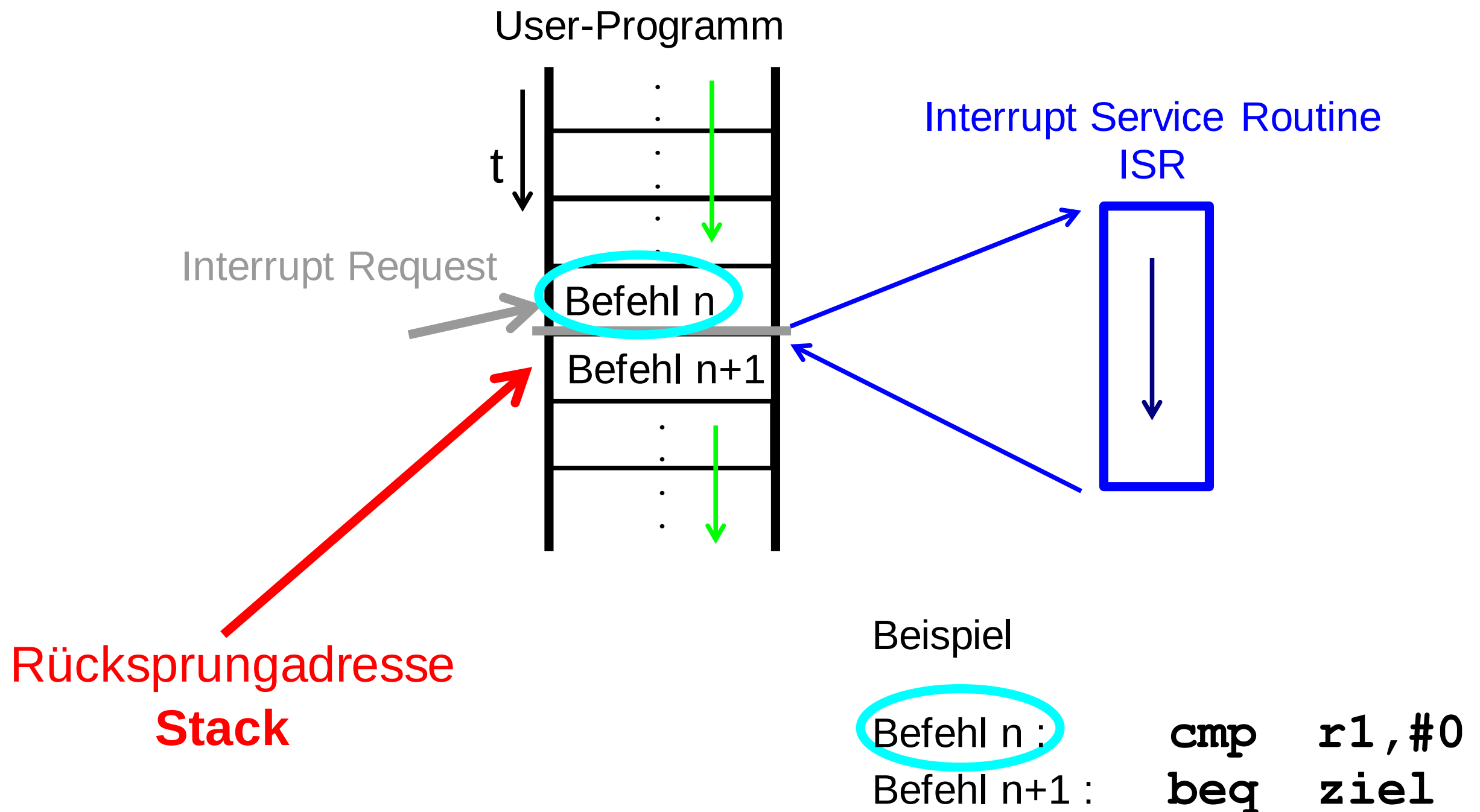


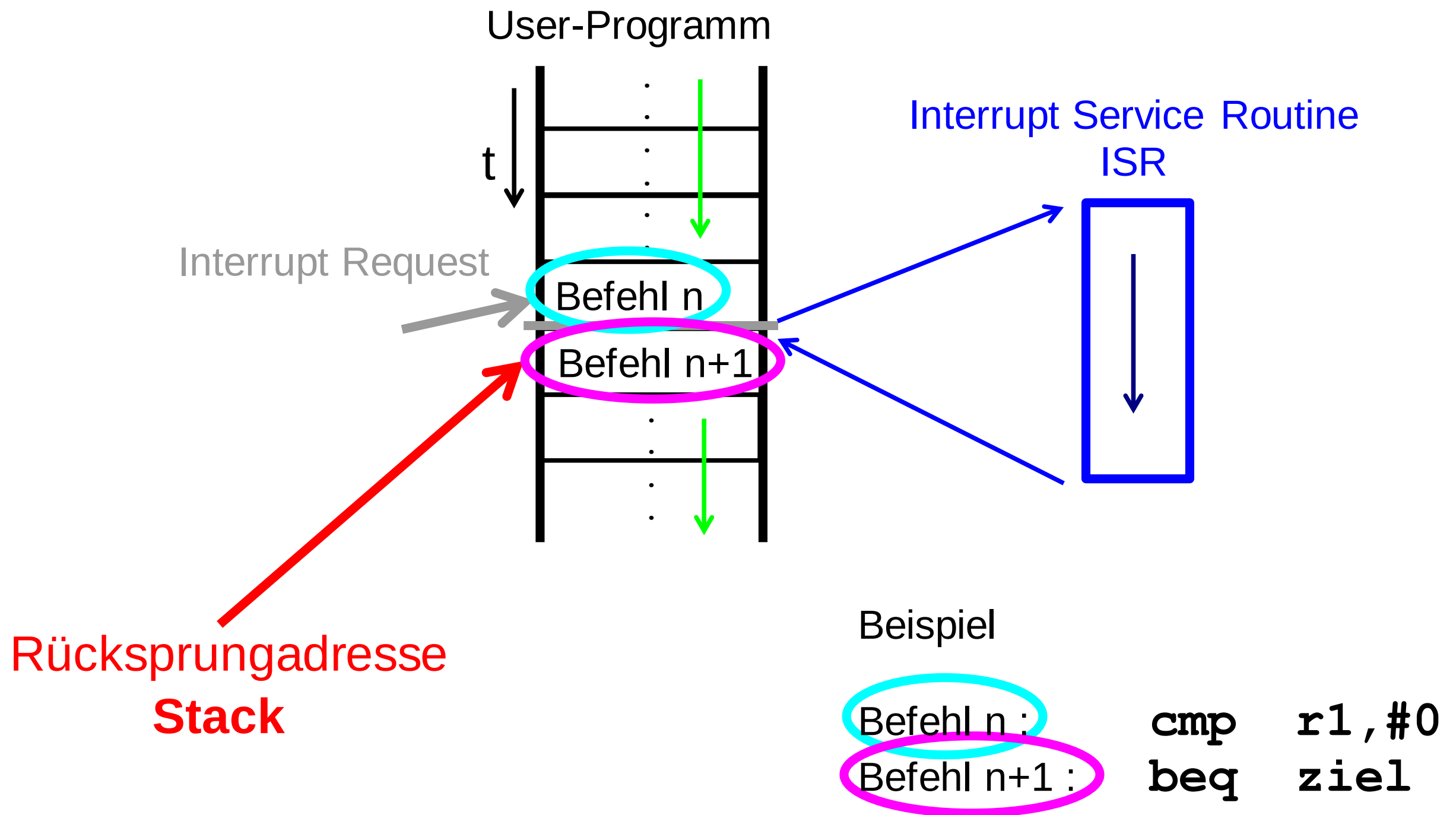


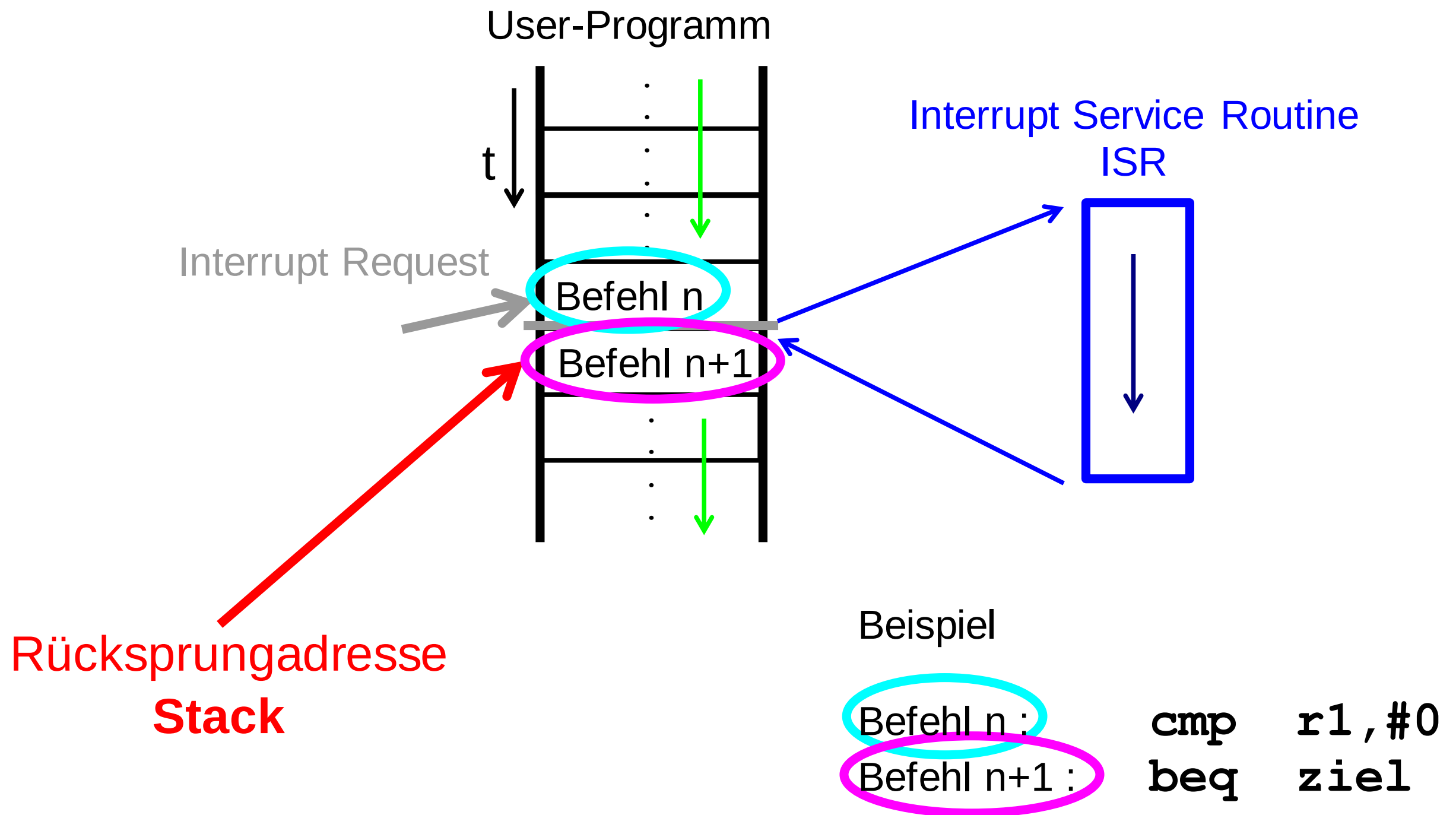








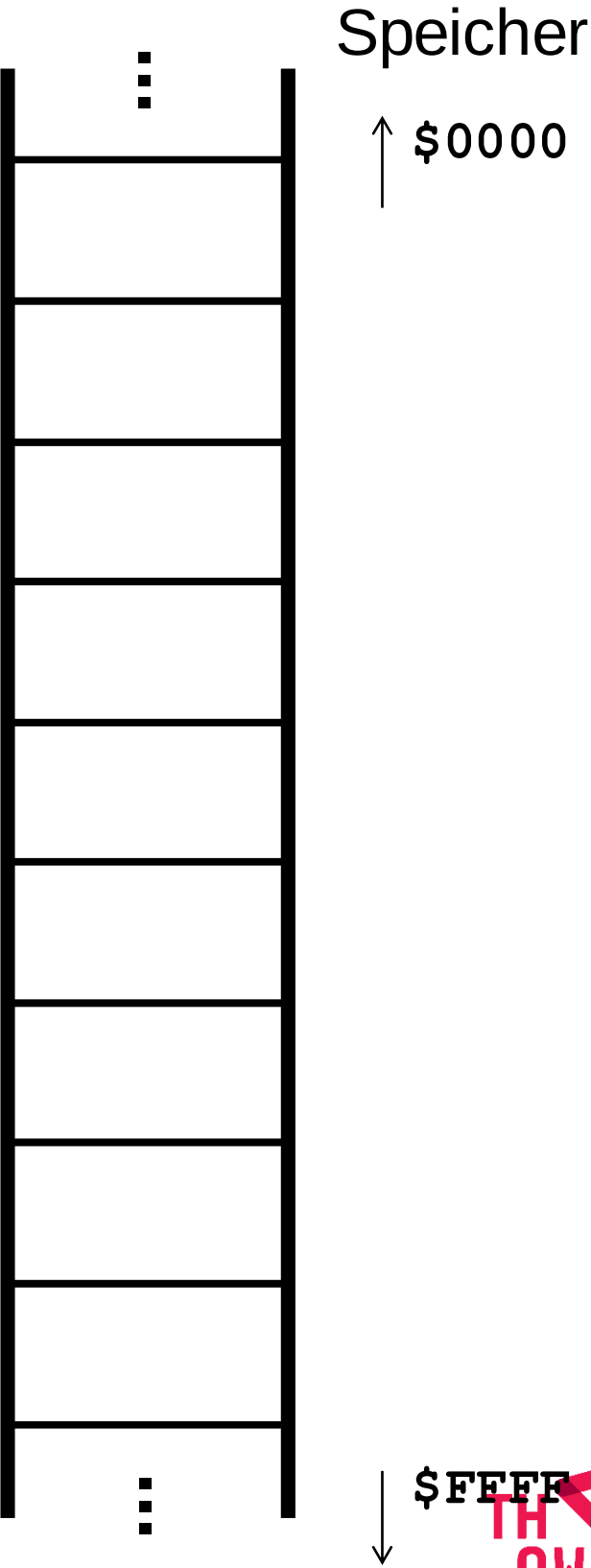






Unterprogramm  
(BL)

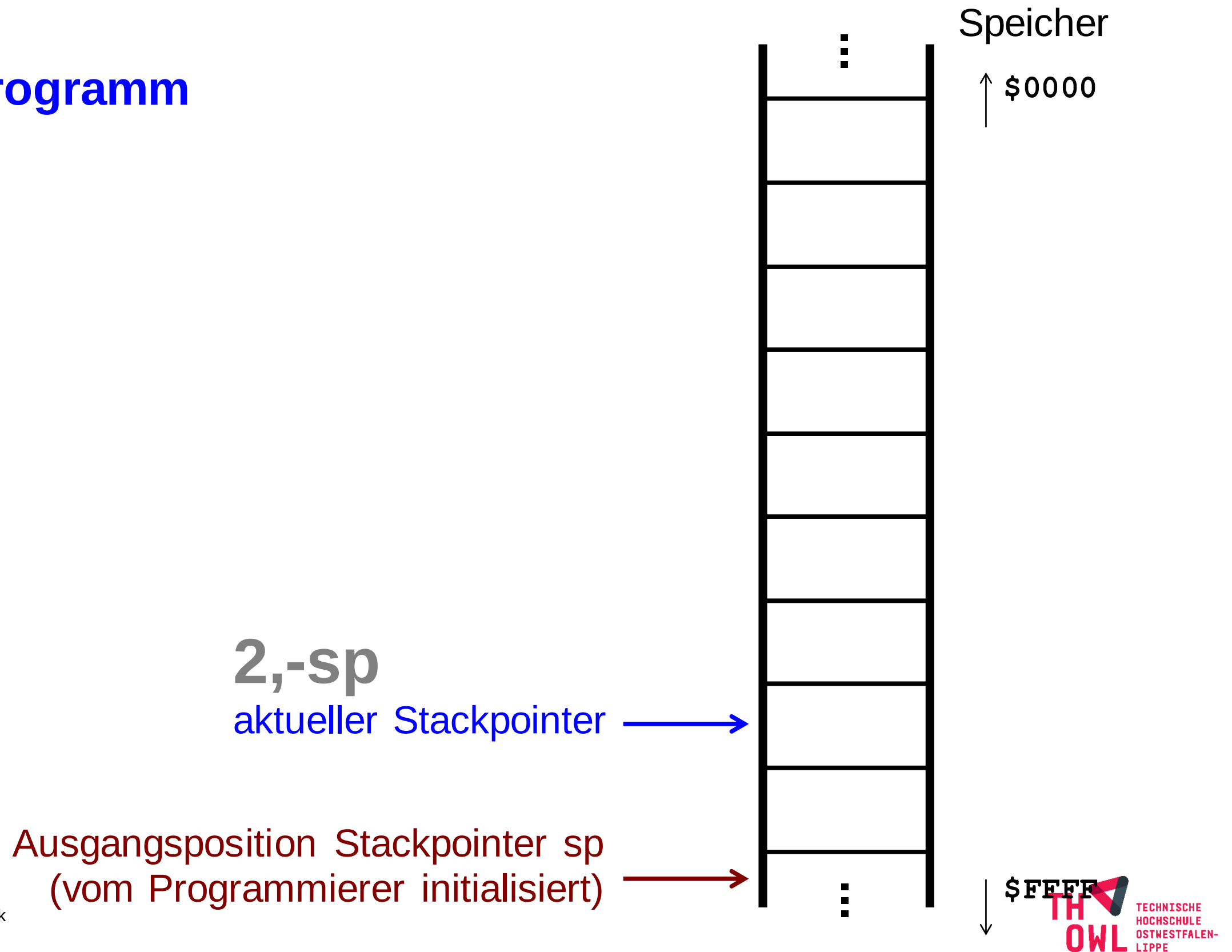
Ausgangsposition Stackpointer sp  
(vom Programmierer initialisiert)



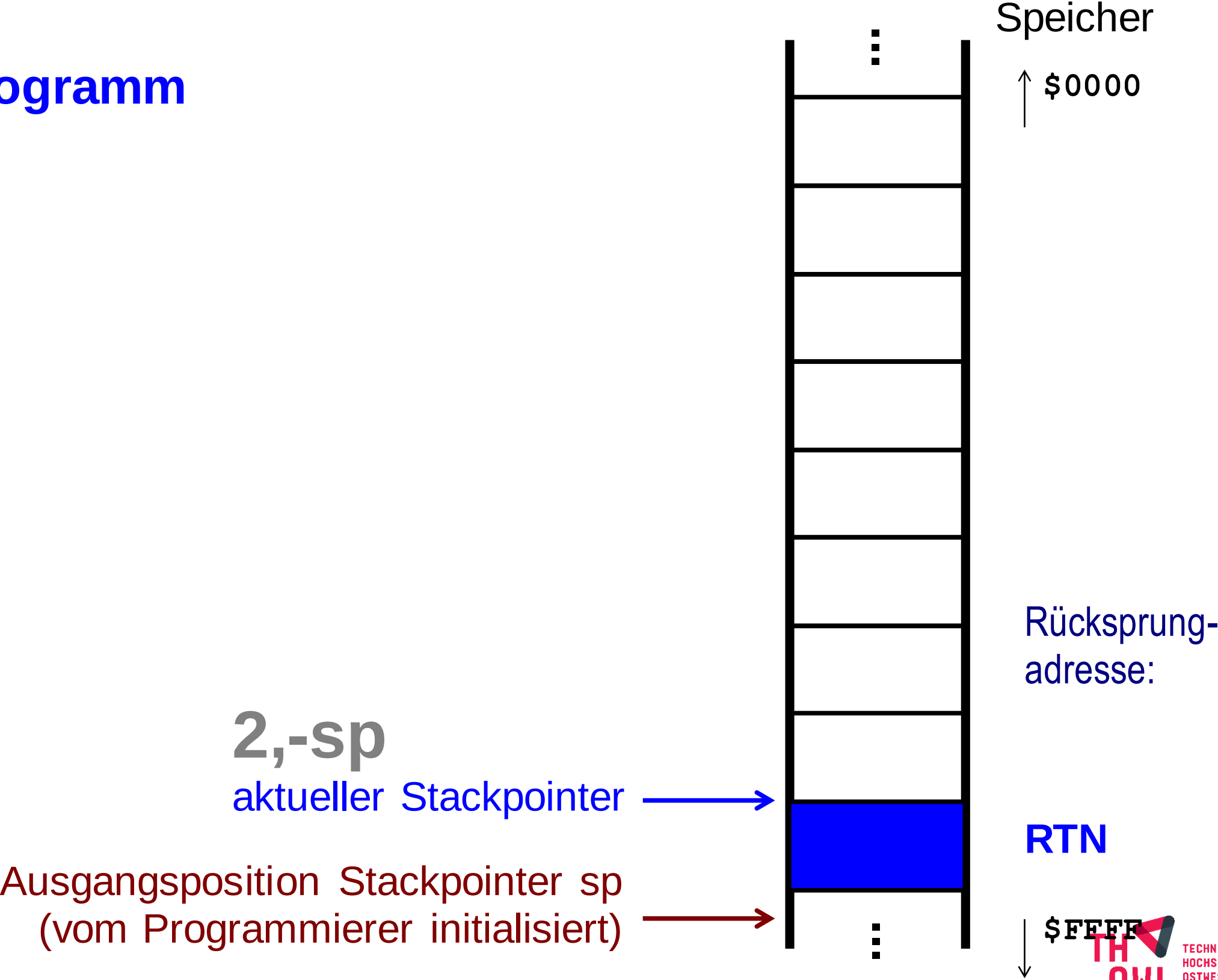


# Ausnahmeverarbeitung (Exceptions)

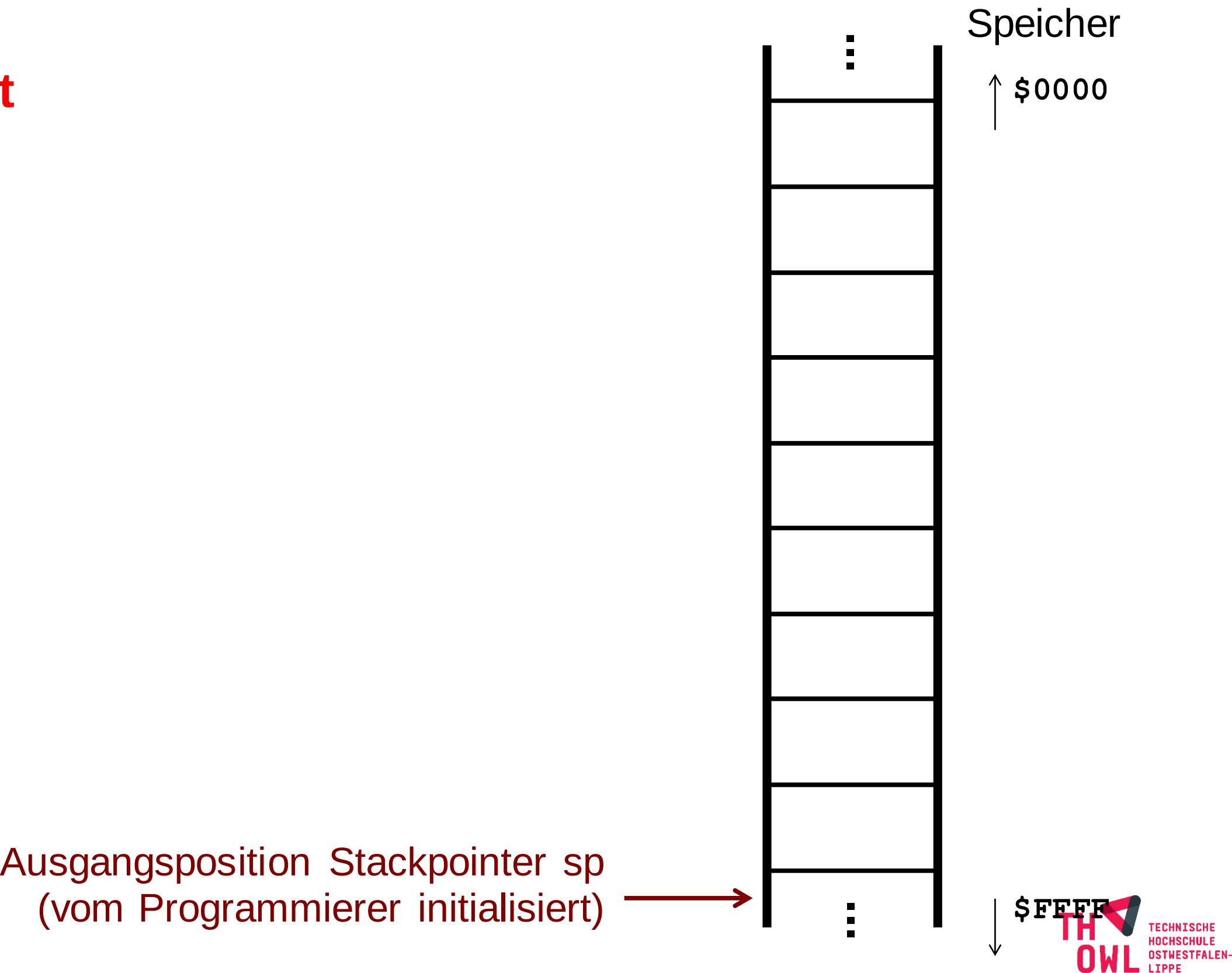
Unterprogramm  
(BL)



Unterprogramm  
(BL)



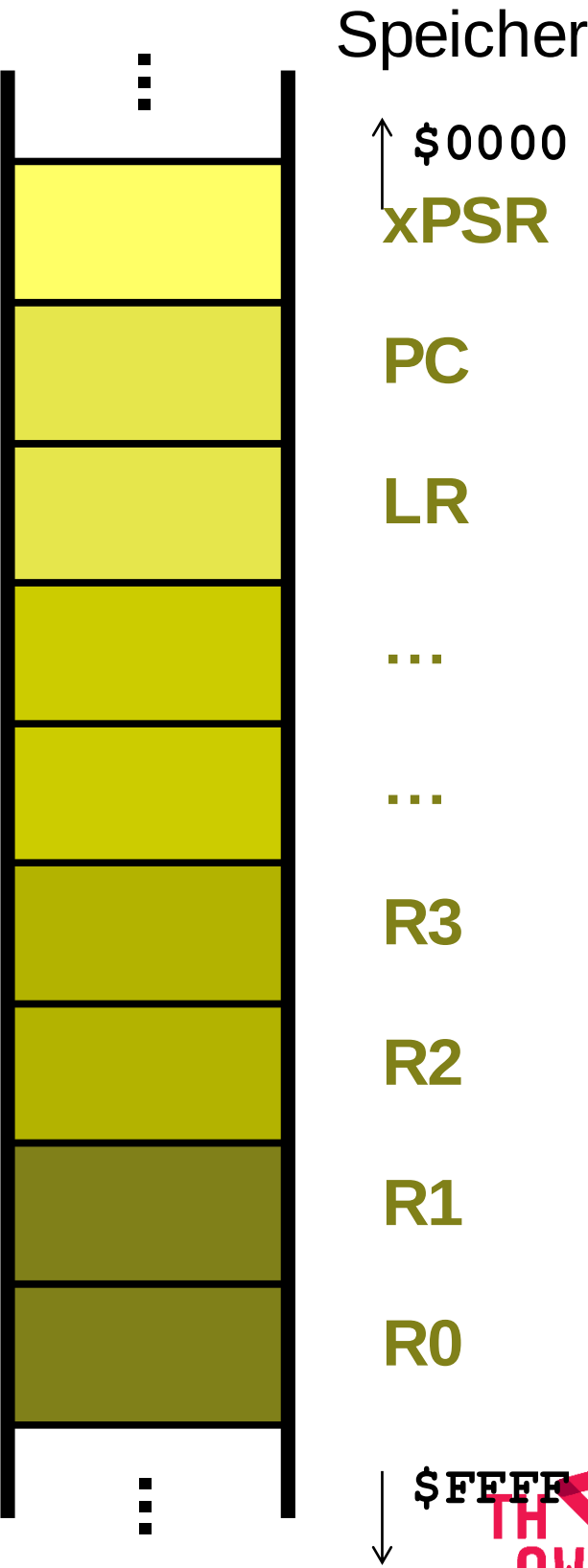
Interrupt



Interrupt

aktueller Stackpointer →

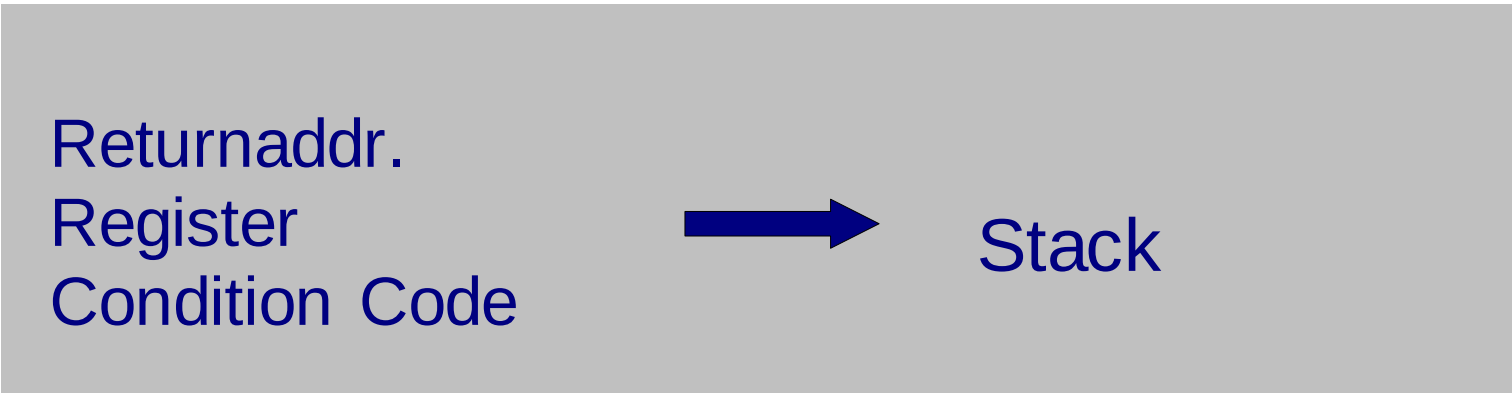
Ausgangsposition Stackpointer sp  
(vom Programmierer initialisiert) →



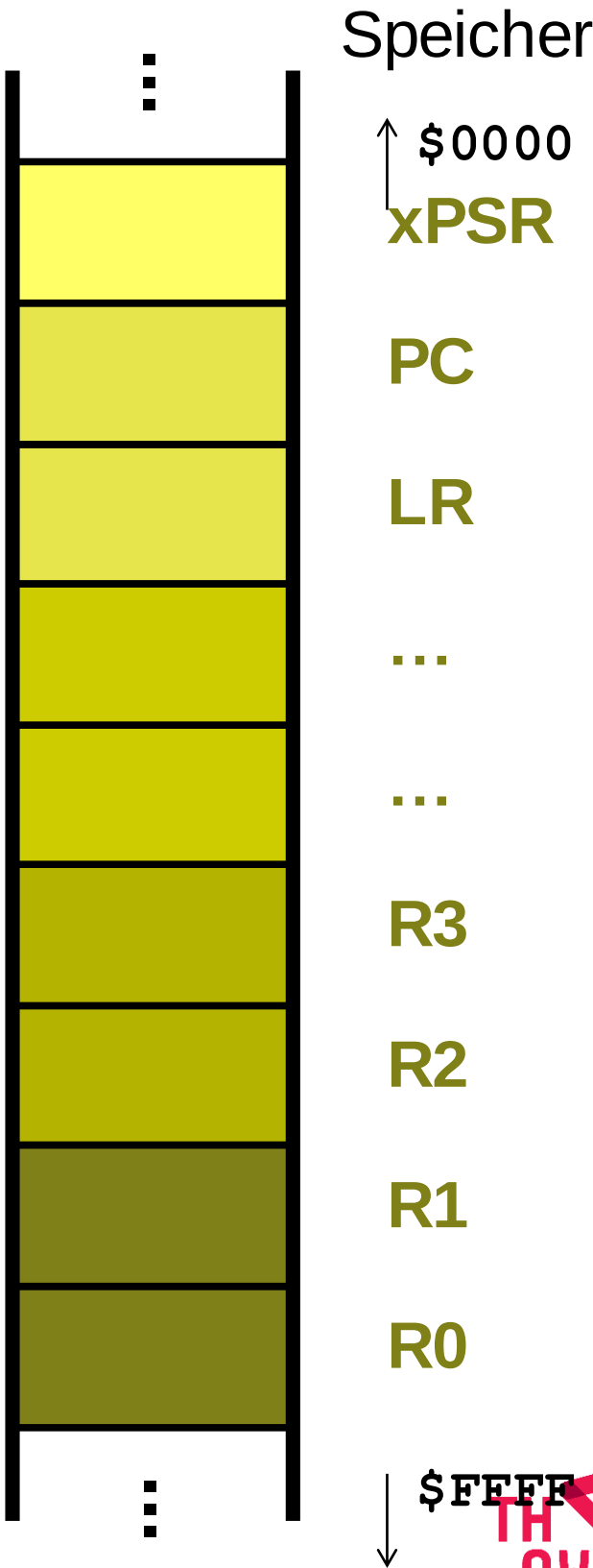
# Ausnahmeverarbeitung (Stack layout)

Interrupt

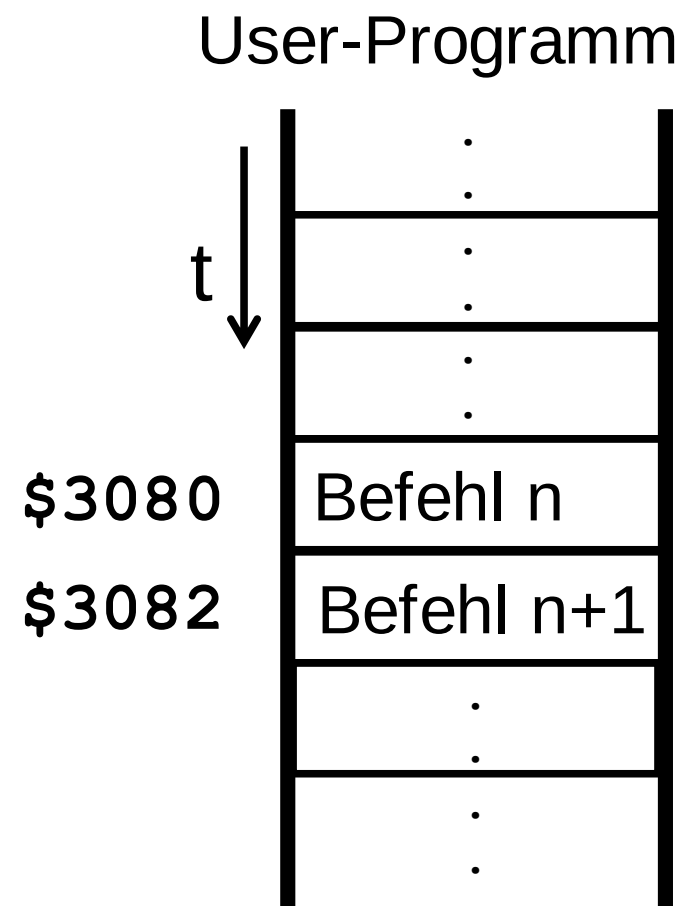
aktueller Stackpointer →



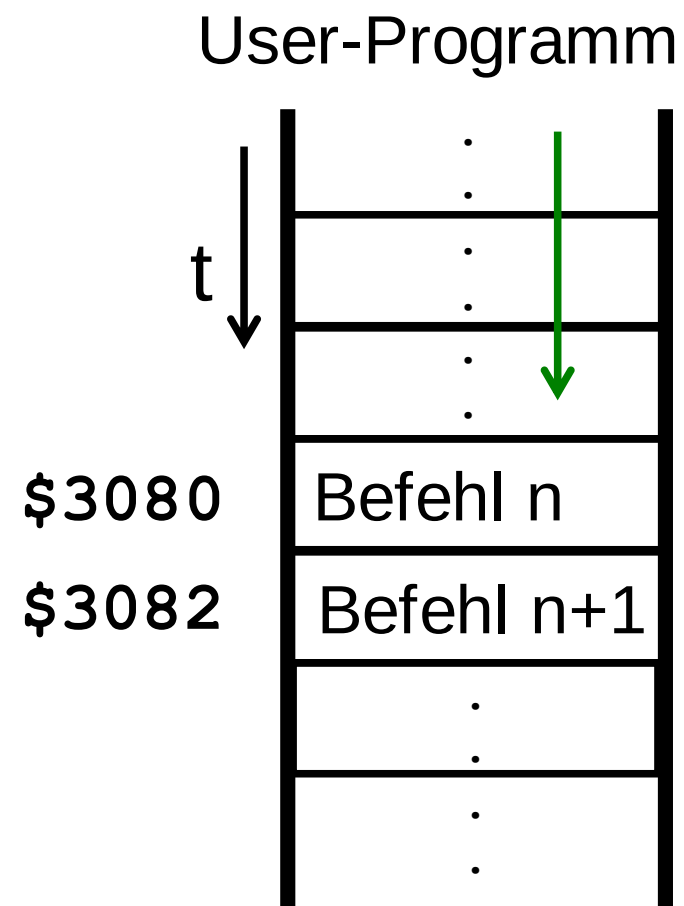
Ausgangsposition Stackpointer sp  
(vom Programmierer initialisiert) →



# Interruptverarbeitung (Beispiel)

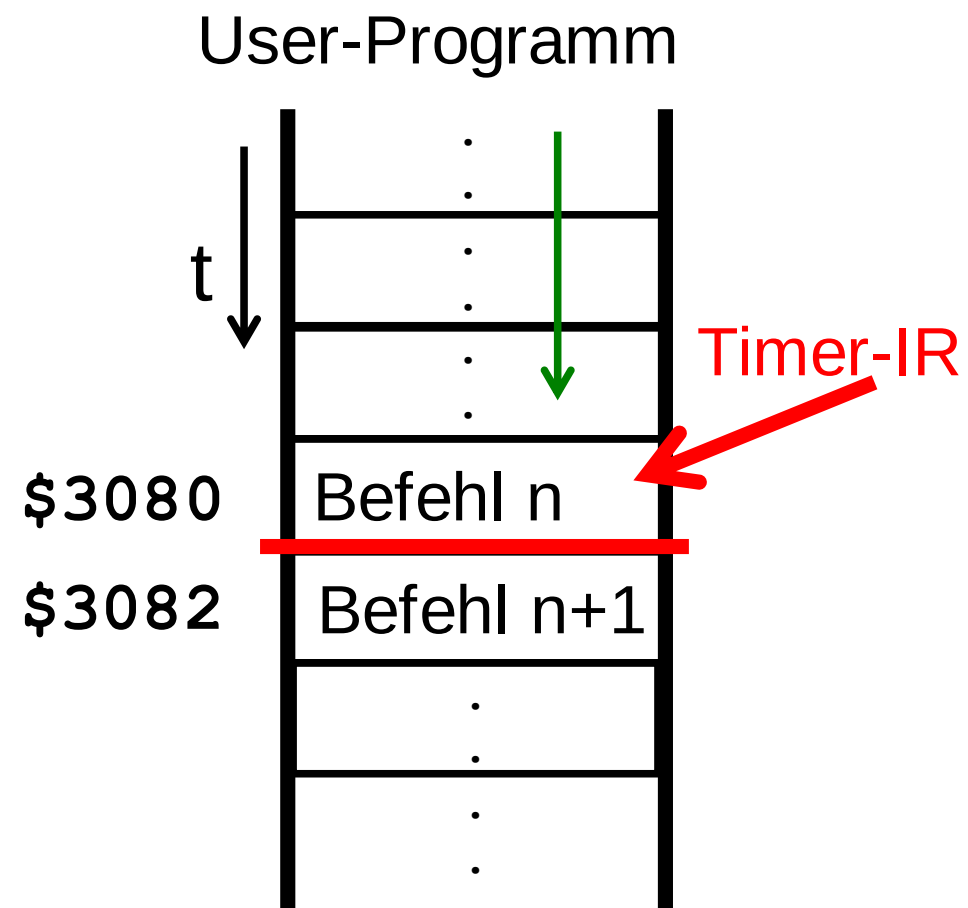


# Interruptverarbeitung (Beispiel)



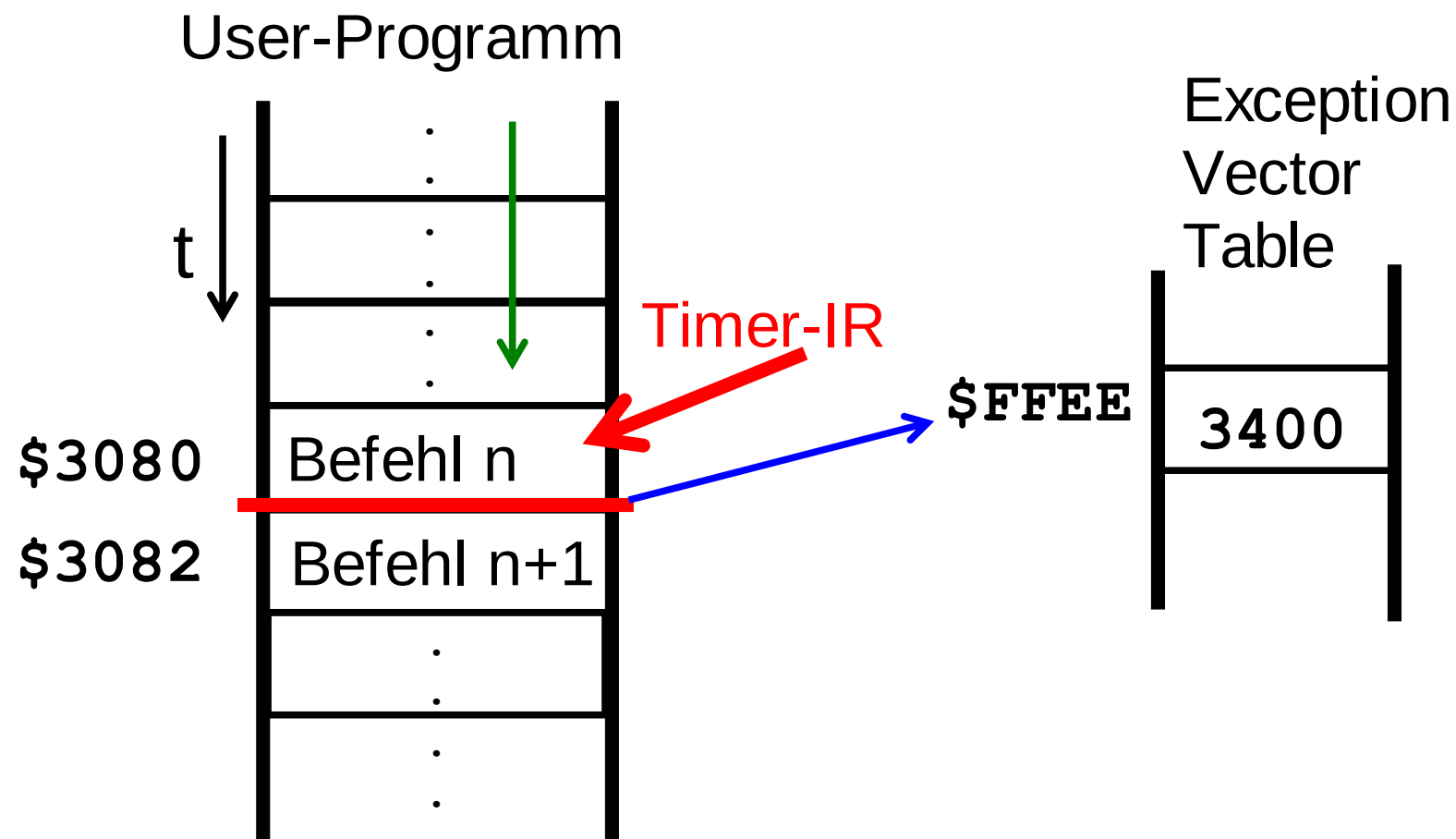


# Interruptverarbeitung (Beispiel)

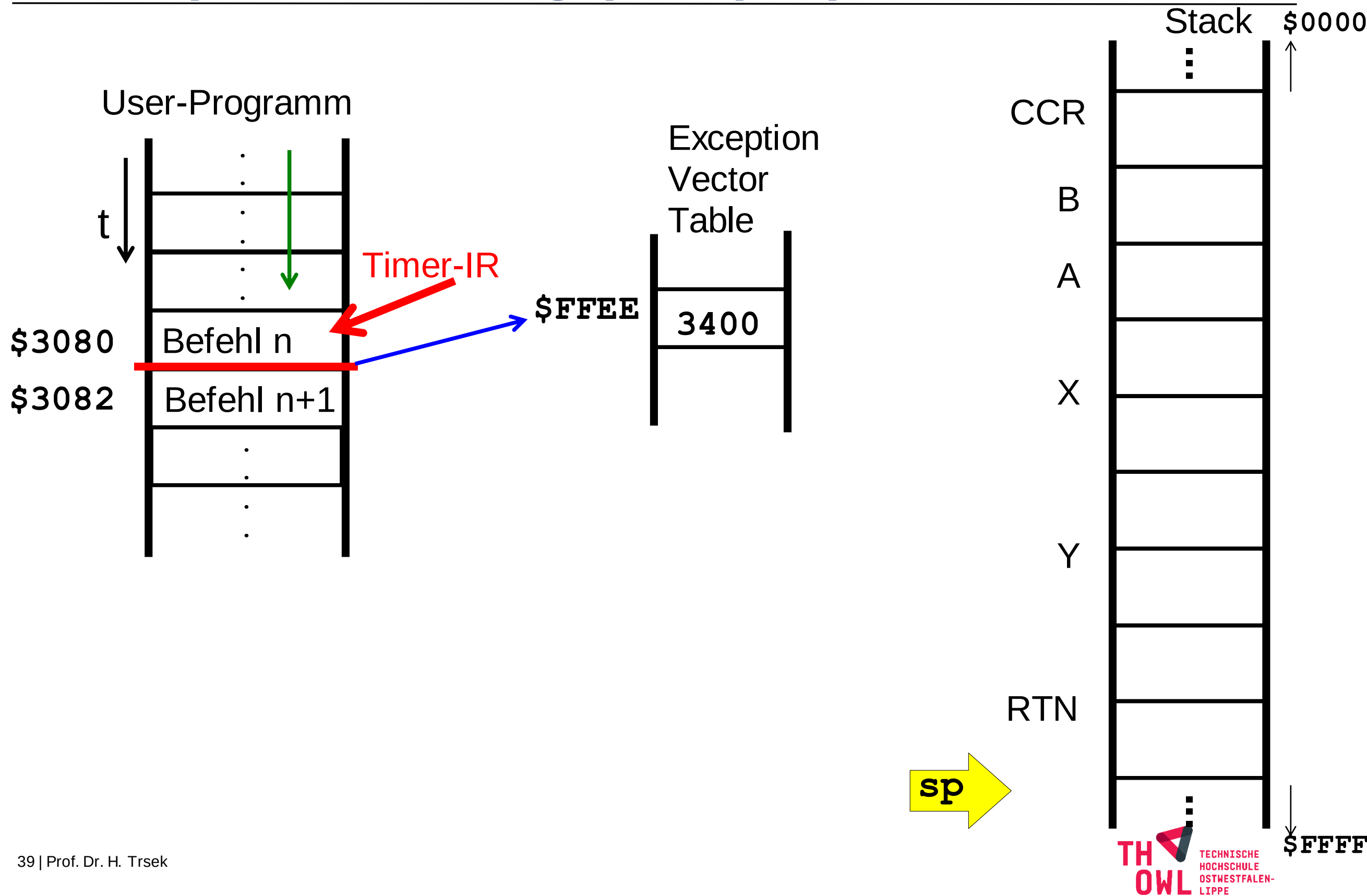




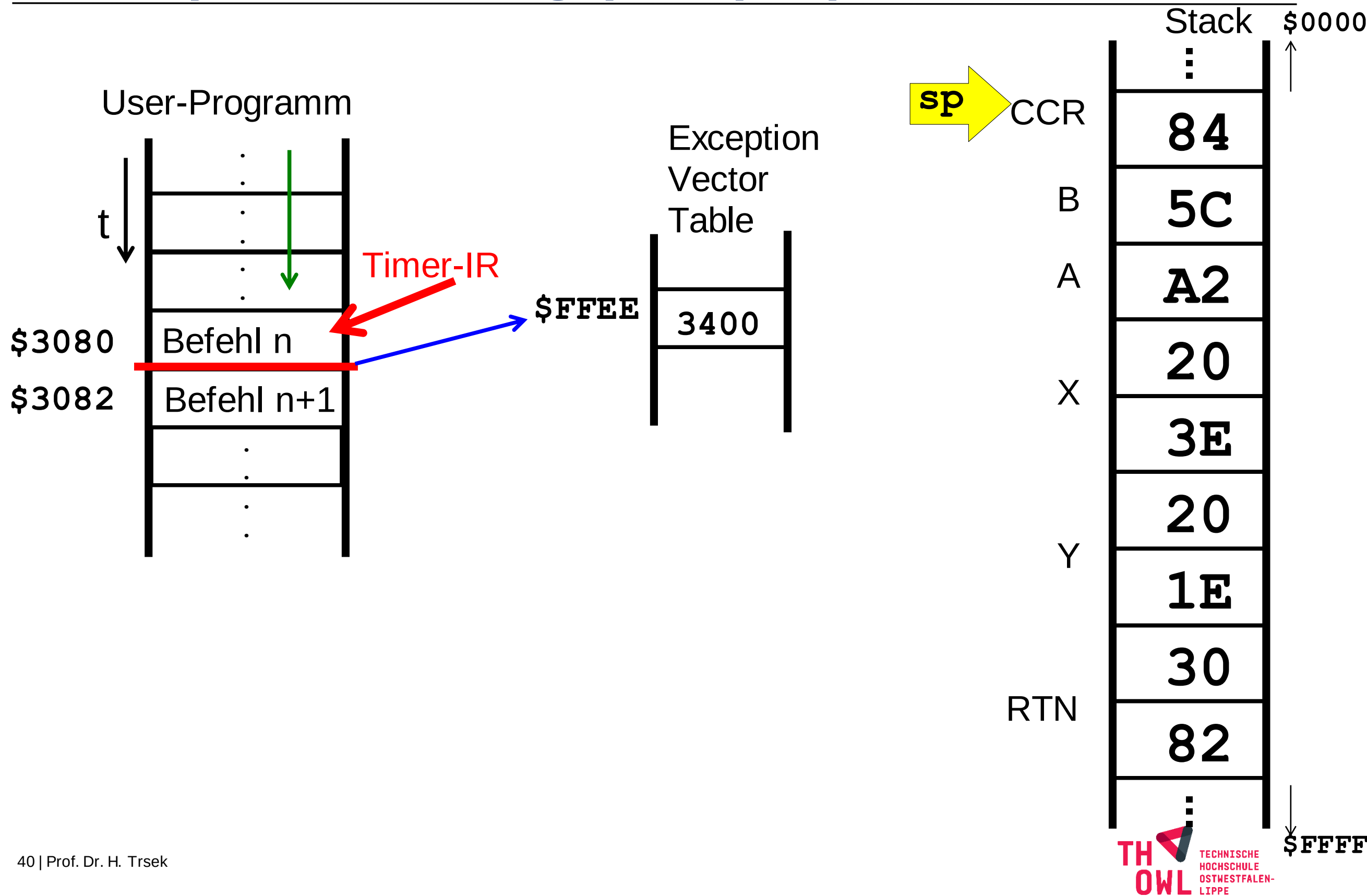
# Interruptverarbeitung (Beispiel)



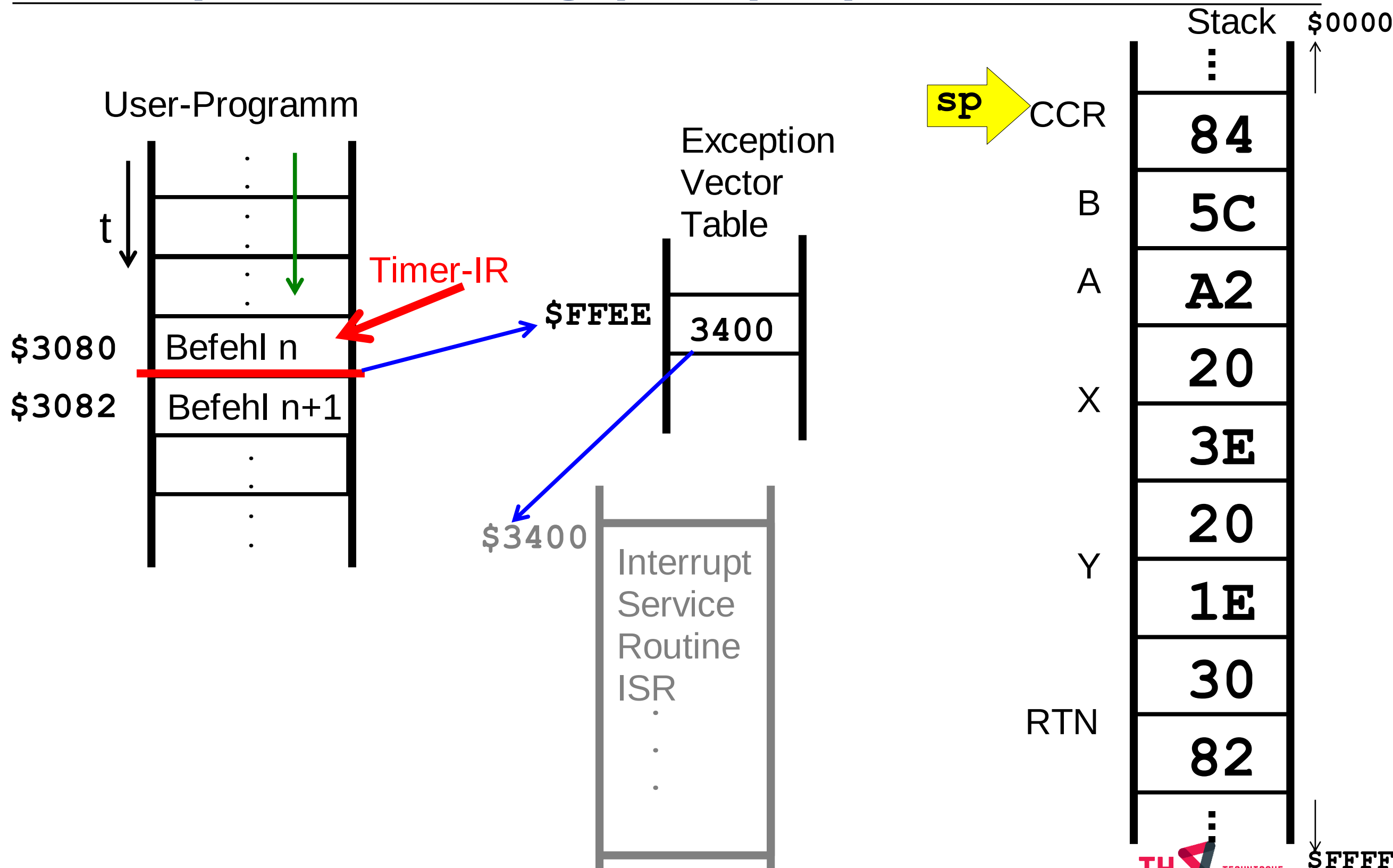
# Interruptverarbeitung (Beispiel)



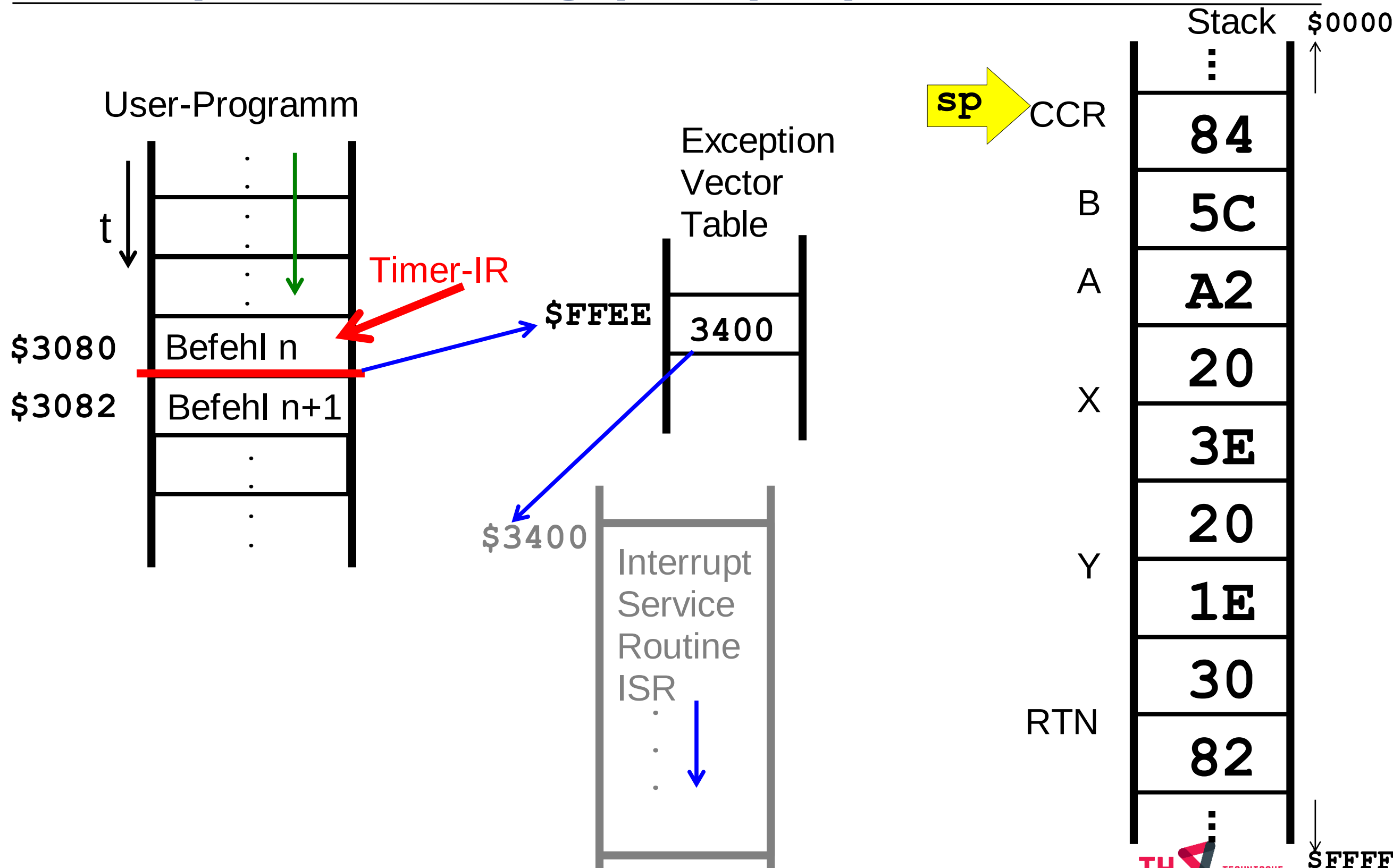
# Interruptverarbeitung (Beispiel)



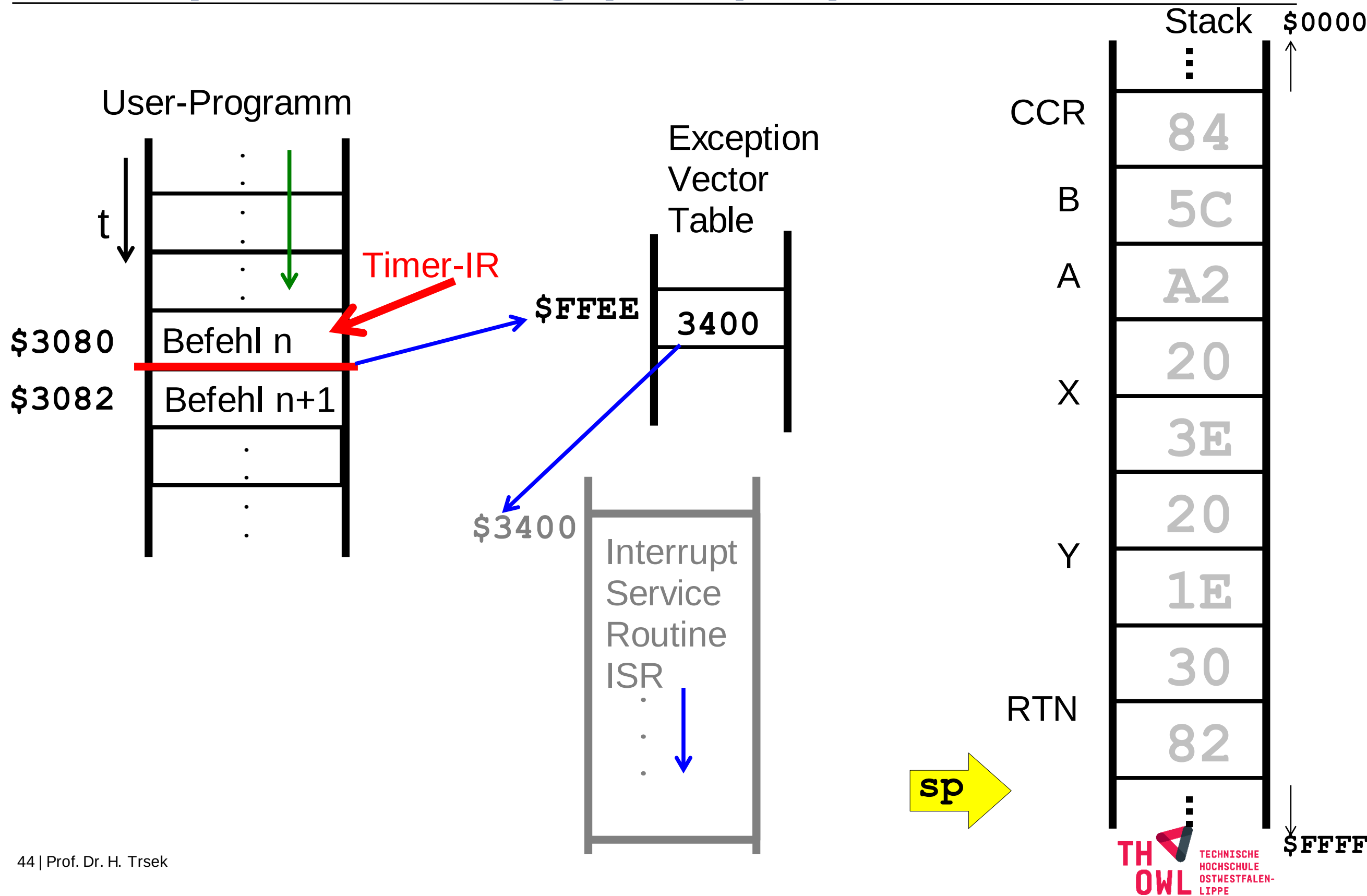
# Interruptverarbeitung (Beispiel)



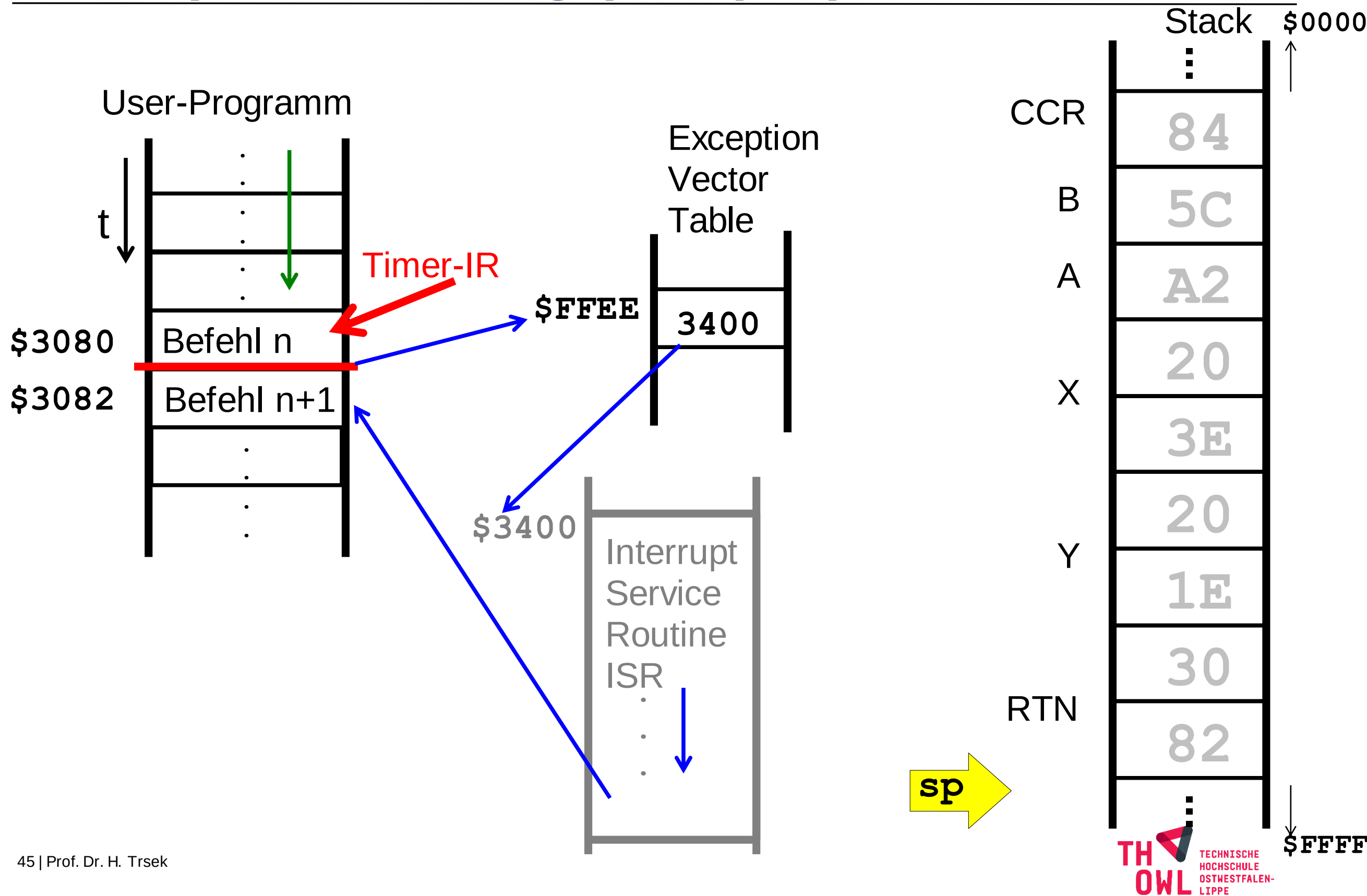
# Interruptverarbeitung (Beispiel)



# Interruptverarbeitung (Beispiel)



# Interruptverarbeitung (Beispiel)





# Interruptverarbeitung (Beispiel)

